

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN
KHAI THÁC DỮ LIỆU VÀ ỨNG DỤNG



BÁO CÁO MÔN HỌC
PROJECT 02 - KHAI THÁC TẬP PHỔ BIẾN

Giảng viên hướng dẫn : GS. Lê Hoài Bắc
ThS. Lê Nhật Nam
ThS. Nguyễn Ngọc Đức
ThS. Nguyễn Thị Thu Hằng

Nhóm thực hiện : Nhóm 9

Lớp : CQ2023/24

Nhóm sinh viên thực hiện :

Bàng Mỹ Linh	23122009
Lại Nguyễn Hồng Thanh	23122018
Phan Huỳnh Châu Thịnh	23122019
Nguyễn Gia Bảo	23122015

Tháng 4 năm 2026

Mục lục

Danh sách bảng	5
Danh sách hình ảnh	7
Danh mục thuật ngữ	9
1 NỀN TẢNG LÝ THUYẾT	10
1.1 Bài toán Frequent Itemset Mining	10
1.1.1 Cơ sở dữ liệu giao dịch	10
1.1.2 Độ hỗ trợ	10
1.1.3 Tập phổ biến	11
1.1.4 Tập đóng và tập tối đại	11
1.1.5 Tính chất Apriori	12
1.2 Phân tích thuật toán RELIM	12
1.2.1 Ý tưởng cốt lõi	12
1.2.2 Cấu trúc dữ liệu chính	14
1.2.3 Mã giả thuật toán	15
1.2.4 Phân tích độ phức tạp	19
1.2.5 Vị trí của Relim trong dòng phát triển FIM	22
2 VÍ DỤ MINH HỌA TAY	25
2.1 Ví dụ 1: Minh họa tổng quát	25
2.1.1 Thiết lập	25
2.1.2 Bước tiền xử lý	25
2.1.3 Thực hiện thuật toán từng bước	27
2.1.4 Tập kết quả	31
2.1.5 Cây đệ quy (Recursion Tree)	31
2.1.6 Kiểm tra chéo	32
2.2 Ví dụ 2: Tình huống đặc biệt - Counter-Only Transactions	32
2.2.1 Mô tả tình huống đặc biệt	32
2.2.2 Thiết lập	33
2.2.3 Xây dựng mảng ban đầu	33
2.2.4 Thực hiện thuật toán từng bước	34

2.2.5 Tập kết quả	36
2.2.6 Kiểm tra chéo	37
3 CÀI ĐẶT	38
3.1 Môi trường và công cụ	38
3.2 Kiến trúc mã nguồn	38
3.2.1 Mô tả các module chính	39
3.3 Cài đặt Level 1: Thuật toán cơ bản	39
3.3.1 Cấu trúc dữ liệu	39
3.3.2 Hàm chính và tiền xử lý	40
3.3.3 Hàm đệ quy chính	41
3.3.4 Kiểm chứng nhanh trên ví dụ	43
3.4 Cài đặt Level 2: Kiểm thử tự động	43
3.4.1 Thiết kế test case	43
3.4.2 Kết quả kiểm thử	43
3.5 Cài đặt Level 3: Tối ưu hóa	44
3.5.1 Mô tả kỹ thuật tối ưu hóa	44
3.5.2 Đo lường hiệu quả tối ưu hóa	44
3.6 Cài đặt Level 4: Xử lý I/O chuẩn SPMF	45
3.6.1 Hàm đọc/ghi file (utils.jl)	45
3.6.2 Giao diện dòng lệnh (main.jl)	45
4 THỰC NGHIỆM VÀ ĐÁNH GIÁ	47
4.1 Tập dữ liệu benchmark	47
4.2 Thực nghiệm bắt buộc	47
4.2.1 Kiểm tra tính đúng đắn (Correctness)	47
4.2.2 Thời gian chạy theo minsup	48
4.2.3 Số lượng frequent itemset theo minsup	50
4.2.4 Sử dụng bộ nhớ	51
4.2.5 Khả năng mở rộng (Scalability)	52
4.2.6 Ảnh hưởng của độ dài giao dịch trung bình	53
4.3 Phân tích kết quả	54
4.3.1 Giải thích kết quả dựa trên lý thuyết	54
4.3.2 So sánh điểm mạnh và điểm yếu so với SPMF	55
4.3.3 Đề xuất hướng tối ưu hóa	56
5 ỨNG DỤNG THỰC TẾ – PHÂN TÍCH GIỎ HÀNG	57

5.1 Giới thiệu bài toán và dataset	57
5.2 Sinh luật kết hợp từ frequent itemset	57
5.2.1 Tham số thực nghiệm	58
5.3 Kết quả: Top-10 luật theo lift	58
5.4 Giải thích ý nghĩa kinh doanh	59
 6 PHÂN CÔNG CÔNG VIỆC	 61
 Tài liệu tham khảo	 63
 Phụ lục	 64

Danh sách bảng

1.1 Ví dụ về cơ sở dữ liệu giao dịch với $\mathcal{I} = \{1, 2, 3, 4, 5\}$	10
1.2 Quy ước ký hiệu trong mã giả	16
1.3 So sánh các thuật toán FIM qua các thể hệ	24
2.1 Cơ sở dữ liệu gốc (Ví dụ 1)	25
2.2 Tần suất các hạng mục và thứ tự sắp xếp	25
2.3 Cơ sở dữ liệu sau sắp xếp ($a < e < c < b < d$)	26
2.4 Mảng A sau khởi tạo - Ví dụ 1	26
2.5 Trạng thái mảng A sau Bước 1 (sau reassign a)	27
2.6 Trạng thái mảng A sau Bước 2 (sau reassign e)	28
2.7 Trạng thái mảng A sau Bước 3 (sau reassign c)	29
2.8 Trạng thái mảng A sau Bước 4 (sau reassign b)	30
2.9 Kiểm tra chéo toàn bộ tập hạng mục (Ví dụ 1, $\sigma = 3$)	32
2.10 Cơ sở dữ liệu gốc và sau tiền xử lý (Ví dụ 2)	33
2.11 Tần suất và thứ tự sắp xếp (Ví dụ 2)	33
2.12 Phân loại giao dịch vào mảng A theo Thuật toán 1	33
2.13 Trạng thái mảng A sau khởi tạo (Ví dụ 2)	33
2.14 Mảng A sau Bước 1 (Ví dụ 2)	34
2.15 Mảng A sau Bước 2 (Ví dụ 2)	35
2.16 Mảng A sau Bước 3 (Ví dụ 2)	36
2.17 Kiểm tra chéo toàn bộ tập hạng mục (Ví dụ 2, $\sigma = 3$)	37
3.1 Môi trường thực nghiệm	38
3.2 Kết quả chạy bộ kiểm thử <code>test_unit.jl</code>	43
3.3 So sánh hiệu năng bản cơ bản (Level 1, <code>counter_only=false</code>) và bản tối ưu (Level 3, <code>counter_only=true</code>)	44
4.1 Tóm tắt tập dữ liệu benchmark	47
4.2 Kết quả kiểm tra tính đúng đắn trên 5 bộ dữ liệu benchmark	48
4.3 Thời gian chạy của RELIM theo ngưỡng <code>minsup</code>	49

4.4 Số lượng frequent itemset theo ngưỡng minsup	50
4.5 Bộ nhớ cấp phát (Allocated Memory) theo bộ dữ liệu	51
4.6 Kết quả thực nghiệm Scalability – Bộ dữ liệu Retail	52
4.7 Ảnh hưởng của AvgLen đến thời gian chạy RELIM (dữ liệu mô phỏng)	53
5.1 Tóm tắt đặc trưng bộ dữ liệu Groceries	57
5.2 Tham số thực nghiệm Market Basket Analysis	58
5.3 Top-10 luật kết hợp theo Lift – Bộ dữ liệu Groceries (minsup= 1%, minconf= 30%)	59
6.1 Phân công công việc nhóm 9 – Project 2	62

Danh sách hình ảnh

1.1	Sơ đồ Venn thể hiện quan hệ bao hàm giữa Tập phổ biến, Tập đóng và Tập tối đại.	12
1.2	So sánh trực quan: FP-Growth dùng cây tiền tố với mạng lưới con trở phức tạp (trái), trong khi RELIM chỉ dùng mảng danh sách liên kết đơn tối giản (phải).	14
1.3	Cấu trúc mảng danh sách liên kết đơn của thuật toán Relim (Đã tinh chỉnh khoảng cách).	15
1.4	Dòng thời gian phát triển của các thuật toán Khai phá tập phổ biến tiêu biểu.	22
2.1	Cây đệ quy của thuật toán RELIM trên Ví dụ 1. Các nhánh con có support $< \sigma$ bị cắt tỉa nên không hiển thị.	31
4.1	Đồ thị 1 – Thời gian chạy theo ngưỡng minsup trên 5 bộ dữ liệu benchmark (RELIM Julia).	49
4.2	Đồ thị 5 – So sánh thời gian chạy: RELIM nhóm (Julia) và SPMF tham chiếu (Java) trên cùng dataset và minsup. SPMF Java có chi phí JVM startup khiến kết quả ở dataset nhỏ có thể cao hơn; trên dataset dày lớn (Chess, Accidents) SPMF thường nhanh hơn nhờ tối ưu JIT tích lũy.	50
4.3	Đồ thị 2 – Số lượng frequent itemsets sinh ra theo ngưỡng minsup trên 5 bộ dữ liệu. Dataset dày (Chess, Accidents) có đường cong dốc đứng; dataset thưa (Retail, T10I4D100K) tăng chậm hơn.	51
4.4	Đồ thị 6 – So sánh bộ nhớ cấp phát giữa bản cơ bản (L1, counter_only=false) và bản tối ưu (L3, counter_only=true). Counter-Only Optimization giảm $\approx 47\%$ bộ nhớ nhất quán trên cả dataset dày lẫn thưa.	52
4.5	Đồ thị 3 – Scalability: Thời gian chạy theo kích thước CSDL (tập con Retail 10–100%). Đường đứt nét đỏ là tham chiếu tuyến tính. Kết quả cho thấy RELIM scale gần tuyến tính từ 25% đến 100%.	53
4.6	Đồ thị 4 – Ảnh hưởng của độ dài giao dịch trung bình (AvgLen) lên thời gian chạy RELIM. Runtime tăng từ 1,60 ms (AvgLen = 5) lên 77,42 ms (AvgLen = 30), tăng ≈ 48 lần khi AvgLen tăng 6 lần.	54

5.1 Biểu đồ Top-10 luật kết hợp mạnh nhất theo Lift – Groceries Dataset (minsup=1%, minconf=30%). Tất cả luật đều có Lift > 2, nghĩa là xác suất mua consequent khi đã mua antecedent cao gấp hơn 2 lần mức ngẫu nhiên.	59
---	-----------

DANH MỤC THUẬT NGỮ

Thuật ngữ tiếng Việt	Thuật ngữ tiếng Anh (Gốc)	Ký hiệu / Ý nghĩa trong thuật toán
Cơ sở dữ liệu giao dịch	Transaction database	$\mathcal{D}$ - Dữ liệu đầu vào, được rút gọn trong bước tiền xử lý.
Giao dịch	Transaction	T - Một tập con các hạng mục.
Tập hạng mục	Item set	X, Y
Độ hỗ trợ tuyệt đối	Support counter / Frequencies	$sup(X)$ - Được lưu tại biến đếm của mỗi phần tử trong mảng danh sách liên kết.
Ngưỡng hỗ trợ tối thiểu	Minimum support	$minsup$ - Điều kiện cốt lõi để cắt tĩa item và gọi đệ quy.
Tập phổ biến	Frequent item set	F - Kết quả đầu ra duy nhất của thuật toán.
Tiền tố	Prefix	P - Tập hợp các item đã xét trong các vòng đệ quy trước đó.
Phân bổ lại	Reassign	Hành động cắt bỏ item đầu tiên và chuyển giao dịch sang danh sách khác nằm bên phải.
Mảng danh sách liên kết đơn	Array of singly linked lists	Cấu trúc dữ liệu tối giản được Relim sử dụng thay cho FP-tree.

Chương 1

NỀN TẢNG LÝ THUYẾT

1.1 Bài toán Frequent Itemset Mining

Phần này trình bày nền tảng lý thuyết hình thức của bài toán khai phá tập phổ biến (Frequent Itemset Mining – FIM), bao gồm các định nghĩa cơ bản, phân loại tập hạng mục và tính chất giải tích quan trọng làm cơ sở cho mọi thuật toán FIM hiện đại.

1.1.1 Cơ sở dữ liệu giao dịch

Định nghĩa 1.1 (Cơ sở dữ liệu giao dịch). Cho $\mathcal{I} = \{i_1, i_2, \dots, i_m\}$ là tập hữu hạn các hạng mục (items) phân biệt. Một giao dịch (transaction) T là một bộ $T = (\text{tid}, X_T)$, trong đó tid là định danh duy nhất của giao dịch và $X_T \subseteq \mathcal{I}$ là tập các hạng mục mà giao dịch đó chứa. Cơ sở dữ liệu giao dịch $\mathcal{D}$ là một đa tập hữu hạn gồm n giao dịch:

$$\mathcal{D} = \{T_1, T_2, \dots, T_n\}, \quad X_{T_k} \subseteq \mathcal{I}, \quad k = 1, \dots, n. \quad (1.1)$$

Khi ngữ cảnh rõ ràng, ta đồng nhất giao dịch T với tập hạng mục X_T của nó và viết $T \subseteq \mathcal{I}$.

Bảng 1.1 trình bày một ví dụ minh họa về cơ sở dữ liệu giao dịch nhỏ sẽ được sử dụng xuyên suốt chương này.

Table 1.1: Ví dụ về cơ sở dữ liệu giao dịch với $\mathcal{I} = \{1, 2, 3, 4, 5\}$

TID	Giao dịch
T_1	$\{1, 3, 4\}$
T_2	$\{2, 3, 5\}$
T_3	$\{1, 2, 3, 5\}$
T_4	$\{2, 5\}$

1.1.2 Độ hỗ trợ

Định nghĩa 1.2 (Độ hỗ trợ tuyệt đối). Độ hỗ trợ tuyệt đối của một tập hạng mục $X \subseteq \mathcal{I}$ trong cơ sở dữ liệu $\mathcal{D}$, ký hiệu $\text{sup}(X)$, là số lượng giao dịch T trong $\mathcal{D}$ chứa X :

$$\text{sup}(X) = |\{T \in \mathcal{D} \mid X \subseteq T\}|. \quad (1.2)$$

Định nghĩa 1.3 (Độ hỗ trợ tương đối). Độ hỗ trợ tương đối của X , ký hiệu $\text{rsup}(X)$, là tỉ lệ giao dịch T chứa X :

$$\text{rsup}(X) = \frac{\text{sup}(X)}{|\mathcal{D}|} \in [0, 1]. \quad (1.3)$$

Độ hỗ trợ tối thiểu σ (min support) do người dùng chỉ định có thể được biểu diễn theo một trong hai dạng: số nguyên tuyệt đối hoặc tỉ lệ thập phân. Trong báo cáo này, ký hiệu σ được dùng thống nhất theo dạng tuyệt đối.

1.1.3 Tập phổ biến

Định nghĩa 1.4 (Tập phổ biến). Một tập hạng mục $X \subseteq \mathcal{I}$ được gọi là tập phổ biến (frequent itemset) với ngưỡng σ khi và chỉ khi:

$$\sup(X) \geq \sigma. \quad (1.4)$$

Tập hợp tất cả các tập phổ biến trong $\mathcal{D}$ được ký hiệu là $\mathcal{F}(\mathcal{D}, \sigma)$, hay $\mathcal{F}$ khi $\mathcal{D}$ và σ đã được xác định rõ từ ngữ cảnh.

$$\mathcal{F} = \{X \mid X \subseteq I \text{ và } \sup(X) \geq \text{minsup}\}. \quad (1.5)$$

Bài toán khai phá tập phổ biến đặt ra yêu cầu tìm ra toàn bộ $\mathcal{F}$, tức là liệt kê đầy đủ mọi tập hạng mục thỏa mãn ngưỡng σ cùng với giá trị độ hỗ trợ tương ứng [1].

1.1.4 Tập đóng và tập tối đại

Trong thực tế, tập $\mathcal{F}$ có thể rất lớn; để thu hẹp biểu diễn, người ta định nghĩa hai lớp con quan trọng sau.

Định nghĩa 1.5 (Tập phổ biến đóng). Một tập phổ biến $X \in \mathcal{F}$ được gọi là tập phổ biến đóng (closed frequent itemset) nếu không tồn tại tập nào bao nó $Y \supsetneq X$ có cùng độ hỗ trợ:

$$X \text{ là tập đóng} \iff \nexists Y \supsetneq X : \sup(Y) = \sup(X). \quad (1.6)$$

Tập tất cả các tập phổ biến đóng được ký hiệu:

$$\mathcal{C} = \{X \mid X \in \mathcal{F} \text{ và } \nexists Y \supsetneq X \text{ sao cho } \sup(X) = \sup(Y)\}. \quad (1.7)$$

Định nghĩa 1.6 (Tập tối đại). Một tập phổ biến $X \in \mathcal{F}$ được gọi là tập phổ biến tối đại (maximal frequent itemset) nếu không tồn tại tập cha nghiêm ngặt $Y \supsetneq X$ nào cũng là tập phổ biến:

$$X \text{ là tập tối đại} \iff \nexists Y \supsetneq X : Y \in \mathcal{F}. \quad (1.8)$$

Gọi $\mathcal{M}$ là tập tất cả các tập phổ biến tối đại:

$$\mathcal{M} = \{X \mid X \in \mathcal{F} \text{ và } \nexists Y \supsetneq X \text{ mà } Y \in \mathcal{F}\}. \quad (1.9)$$

Mệnh đề 1.1 (Quan hệ phân cấp). Ký hiệu $\mathcal{F}_{\max}$ và $\mathcal{F}_{\text{cl}}$ lần lượt là tập hợp các tập tối đại và tập đóng trong $\mathcal{D}$. Khi đó:

$$\mathcal{F}_{\max} \subseteq \mathcal{F}_{\text{cl}} \subseteq \mathcal{F}. \quad (1.10)$$

Tập đóng và tập tối đại mang lại biểu diễn gọn hơn cho $\mathcal{F}$: từ $\mathcal{F}_{\text{cl}}$ có thể phục hồi đầy đủ độ hỗ trợ của mọi tập phổ biến, còn $\mathcal{F}_{\max}$ chỉ cho biết các tập phổ biến lớn nhất (theo quan hệ bao hàm) mà không lưu lại thông tin về độ hỗ trợ của các tập con.

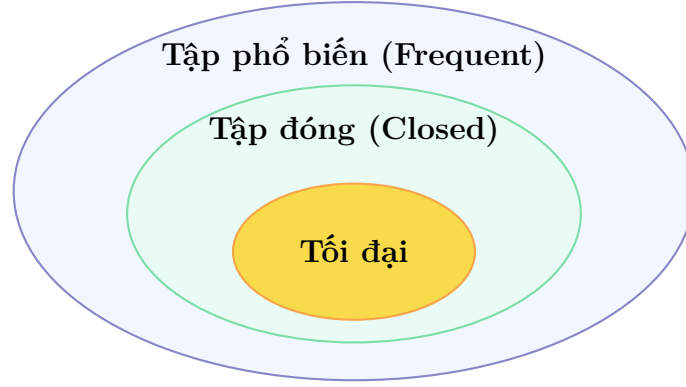


Figure 1.1: Sơ đồ Venn thể hiện quan hệ bao hàm giữa Tập phổ biến, Tập đóng và Tập tối đại.

1.1.5 Tính chất Apriori

Tính chất sau đây là nền tảng lý thuyết cho phép cắt tĩa không gian tìm kiếm trong hầu hết các thuật toán FIM.

Mệnh đề 1.2 (Tính đơn điệu giảm của độ hỗ trợ – Anti-monotonicity). Với mọi $Y \subseteq X \subseteq \mathcal{I}$, ta có:

$$\sup(X) \leq \sup(Y). \quad (1.11)$$

Từ đó suy ra hai hệ quả quan trọng:

- Nếu tập X là phổ biến thì mọi tập con $Y \subseteq X$ cũng phổ biến.
- Nếu tập X không phổ biến thì mọi tập cha $Z \supseteq X$ cũng không phổ biến.

Proof. Xét bất kỳ giao dịch $T \in \mathcal{D}$. Vì $Y \subseteq X$, điều kiện $X \subseteq T$ kéo theo $Y \subseteq T$. Do đó:

$$\{T \in \mathcal{D} \mid X \subseteq T\} \subseteq \{T \in \mathcal{D} \mid Y \subseteq T\}. \quad (1.12)$$

Số phần tử về trái không vượt quá số phần tử về phải; áp dụng Định nghĩa (1.2), ta thu được $\sup(X) \leq \sup(Y)$, điều phải chứng minh. $\square$

Tính chất này cho phép bất kỳ thuật toán FIM nào loại bỏ toàn bộ các tập cha của một tập không phổ biến mà không cần liệt kê, từ đó giảm đáng kể số lượng ứng viên cần xét.

1.2 Phân tích thuật toán RELIM

1.2.1 Ý tưởng cốt lõi

Thuật toán Relim (*Recursive Elimination*) được Borgelt đề xuất năm 2005 [1] với triết lý giữ mọi thứ đơn giản (*Keeping Things Simple*). Relim kế thừa cơ chế đệ quy của FP-Growth và H-Mine, nhưng loại bỏ hoàn toàn các cấu trúc dữ liệu phức tạp như cây tiền tố (FP-tree) hay cấu trúc siêu liên kết (H-struct): toàn bộ thuật toán có thể được cài đặt trong một hàm đệ quy duy nhất.

Ý tưởng cốt lõi của Relim có thể hiểu trực giác như sau. Hình dung không gian tất cả các tập phổ biến cần tìm như một cây con liệt kê tập con. Thay vì xây dựng cây này theo chiều rộng và sinh hàng loạt ứng viên như Apriori, Relim đặt câu hỏi đơn giản hơn:

Với mỗi hạng mục i , có những tập phổ biến nào chứa i ? Nếu trả lời được câu hỏi này cho từng i , ta đã liệt kê được toàn bộ tập phổ biến.

Để tránh đếm trùng (một tập có thể chứa nhiều hạng mục), Relim áp dụng nguyên lý *chia theo hạng mục đầu tiên*: cố định một thứ tự duyệt trên các hạng mục (theo tần suất *tăng dần*); với mỗi hạng mục i , chỉ tìm các tập phổ biến mà i là hạng mục *đầu tiên* (theo thứ tự đã cố định) xuất hiện trong tập đó. Khi đã xử lý xong tất cả tập có i ở đầu, hạng mục i được *loại trừ* hoàn toàn khỏi cơ sở dữ liệu; thuật toán chuyển sang hạng mục kế tiếp. Đây chính là cơ chế “*recursive elimination*” đặt tên cho thuật toán.

Tính đúng đắn của Relim dựa trên hai quan sát đơn giản:

- (i) Mọi tập phổ biến $X \neq \emptyset$ đều có một hạng mục đầu tiên duy nhất theo thứ tự đã chọn. Do đó, nếu lần lượt liệt kê các tập theo từng hạng mục đầu tiên, ta liệt kê được mỗi tập *đúng một lần*, không trùng không thiếu.
- (ii) Mọi tập phổ biến chứa i làm hạng mục đầu tiên đều có dạng $\{i\} \cup Y$ với Y là một tập phổ biến trong *cơ sở dữ liệu điều kiện* của i - tức tập các giao dịch chứa i , sau khi đã loại i và các hạng mục đứng trước i theo thứ tự duyệt. Việc tìm các Y này lại chính là bài toán FIM ban đầu trên một CSDL nhỏ hơn, nên có thể giải đệ quy bằng cùng một thủ tục.

Cộng với tính chất Apriori (Mệnh đề 1.2) để cắt tĩa, ta thu được toàn bộ tập phổ biến mà không cần sinh ứng viên hay quét lại cơ sở dữ liệu nhiều lần.

Thứ tự duyệt là một lựa chọn thiết kế tự do, nhưng có ảnh hưởng quyết định đến hiệu năng. Borgelt phân tích [1]: nếu duyệt theo tần suất *tăng dần*, hạng mục được xử lý sớm là hạng mục *ít phổ biến nhất*, nên CSDL điều kiện của nó nhỏ; các CSDL điều kiện về sau cũng được thu nhỏ tích lũy vì hạng mục đã xử lý bị loại bỏ. Thực nghiệm cho thấy thứ tự *tăng dần* nhanh hơn rõ rệt so với thứ tự ngẫu nhiên, và đặc biệt nhanh hơn thứ tự *giảm dần* [1, Section 2.1].

So với Apriori [2], thuật toán duyệt không gian tìm kiếm theo *chiều rộng* (level-wise), ở vòng lặp thứ k sinh tập ứng viên kích thước k từ các tập phổ biến kích thước $k - 1$, sau đó quét lại toàn bộ CSDL để đếm support. Hai nhược điểm cố hữu: (i) số lần quét CSDL bằng kích thước tập phổ biến dài nhất; (ii) số ứng viên có thể bùng nổ theo hàm mũ trước khi cắt tĩa. Relim hoàn toàn loại bỏ pha sinh ứng viên: nó duyệt theo *chiều sâu* (depth-first), chỉ cần quét CSDL gốc *một lần* để đếm tần suất ban đầu; các bước tiếp theo thao tác trực tiếp trên các CSDL điều kiện.

FP-Growth [3] cũng theo lối đệ quy không sinh ứng viên, nhưng nén CSDL thành cây tiền tố FP-tree để chia sẻ phần đầu chung giữa các giao dịch; với mỗi hạng mục, nó dựng lại một *cây điều kiện* (conditional FP-tree). H-Mine [4] thay cây bằng cấu trúc H-struct gồm các siêu liên kết. Cả hai cấu trúc đều *phải tái cấu trúc* mỗi khi đi sâu một tầng đệ quy - đó là chi phí đáng kể trên các tầng đệ quy sâu.

Relim giữ nguyên ý tưởng đệ quy này nhưng dùng *cấu trúc dữ liệu tối giản*: chỉ một **mảng các danh sách liên kết đơn** (xem Mục 1.2.2), trong đó các giao dịch được *phân loại theo hạng mục đầu tiên*. Khi đi sâu, thuật toán chỉ *sao chép con trỏ* sang mảng mới chứ không tái dựng cấu trúc phức tạp. Cái giá phải trả là Relim không hưởng được lợi ích chia sẻ tiền tố như FP-tree (nên chậm hơn FP-Growth trên dữ liệu dày), nhưng đổi lại nhận được sự đơn giản về cài đặt, ít chi phí khởi tạo ở mỗi tầng đệ quy, và tiêu thụ bộ nhớ thấp - đặc biệt phù hợp với dữ liệu thưa [1].

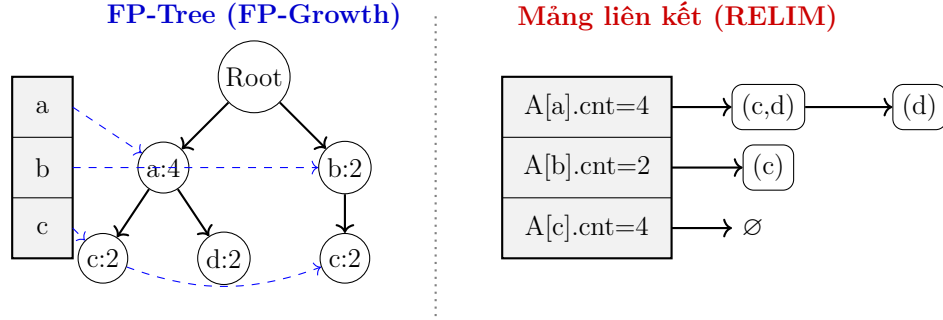


Figure 1.2: So sánh trực quan: FP-Growth dùng cây tiền tố với mạng lưới con trỏ phức tạp (trái), trong khi RELIM chỉ dùng mảng danh sách liên kết đơn tối giản (phải).

1.2.2 Cấu trúc dữ liệu chính

Khác với các thuật toán đương thời như FP-Growth (sử dụng cây tiền tố FP-tree) hay H-Mine (sử dụng H-struct), thuật toán Relim thực hiện toàn bộ quá trình khai phá trực tiếp trên cơ sở dữ liệu giao dịch nhờ một cấu trúc tối giản: **mảng các danh sách liên kết đơn** (array of singly linked lists) [1].

Mỗi giao dịch được lưu trữ dưới dạng một mảng số nguyên, trong đó mỗi phần tử là định danh số nguyên của một hạng mục. Sau bước tiền xử lý, các hạng mục trong mỗi giao dịch được sắp xếp theo thứ tự tần suất tăng dần (hạng mục ít phổ biến hơn đứng trước).

Thuật toán duy trì một mảng một chiều A có k phần tử, với k là số hạng mục phổ biến. Mỗi phần tử $A[i]$ đại diện cho hạng mục thứ i và lưu trữ hai trường: (i) một *biến đếm* $A[i].cnt$ ghi nhận tổng số giao dịch chứa hạng mục đó trong cơ sở dữ liệu hiện tại; và (ii) một *con trỏ đầu danh sách* $A[i].head$ trỏ đến phần tử đầu tiên của danh sách liên kết tương ứng.

Mỗi mắt xích trong danh sách liên kết chứa hai trường: một con trỏ đến mắt xích kế tiếp (*successor pointer*) và một con trỏ vào phần nội dung còn lại của giao dịch (*transaction pointer*).

Khi một giao dịch được đưa vào danh sách $A[i]$ (dựa trên hạng mục đứng đầu là i), hạng mục i được “ẩn đi” bằng cách tăng con trỏ giao dịch lên một vị trí (*pointer increment*). Thông tin về i không bị mất vì nó đã được ngầm định bởi vị trí i của danh sách trong mảng.

Ngoài ra, thuật toán áp dụng tối ưu hóa Counter-Only: nếu sau khi ẩn hạng mục đứng đầu, giao dịch còn lại chỉ gồm đúng một hạng mục j , thuật toán không cấp phát mắt xích danh sách mà chỉ tăng $A[j].cnt$ lên một đơn vị. Tối ưu hóa này loại bỏ hoàn toàn việc cấp phát bộ nhớ động cho các giao dịch ngắn, giảm đáng kể chi phí quản lý bộ nhớ khi đi sâu vào các tầng đệ quy.

Hình 1.3 minh họa trạng thái khởi tạo của mảng danh sách cho một tập dữ liệu ví dụ, trong đó các mũi tên liên biểu diễn con trỏ danh sách, còn mũi tên đứt biểu diễn con trỏ đến nội dung giao dịch sau khi đã ẩn hạng mục đứng đầu.

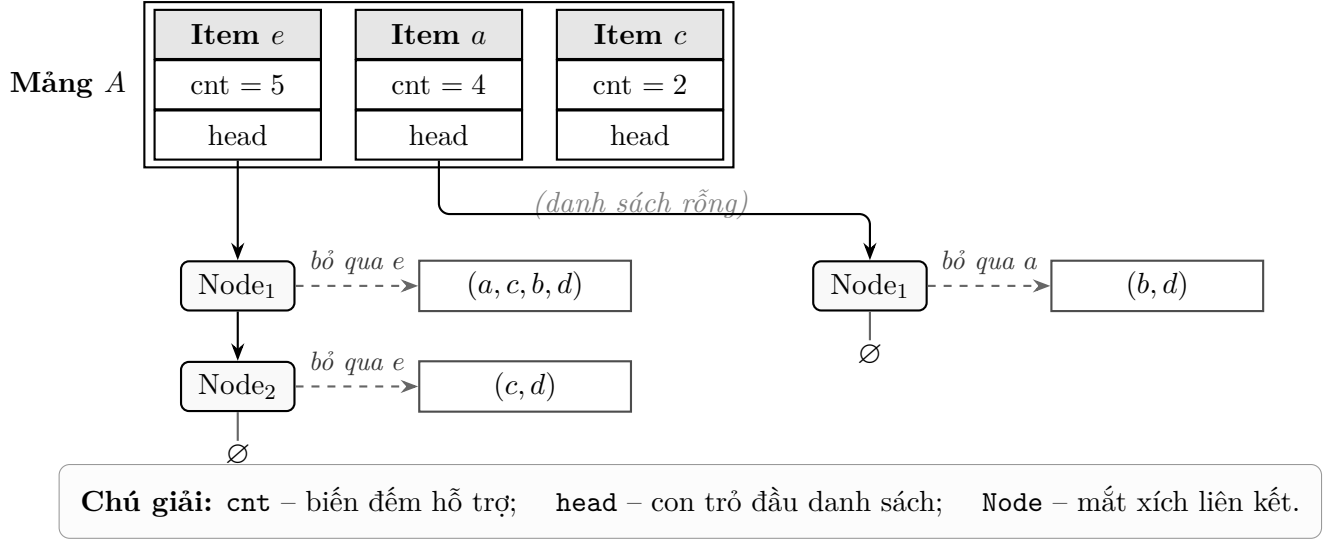


Figure 1.3: Cấu trúc mảng danh sách liên kết đơn của thuật toán Relim (Đã tinh chỉnh khoảng cách).

Nhờ không phải xây dựng lại cấu trúc dữ liệu mới (như tách nhánh trong FP-tree) sau mỗi bước đệ quy, Relim tiết kiệm đáng kể tài nguyên bộ nhớ và chi phí khởi tạo, đặc biệt ở các tầng đệ quy sâu khi kích thước cơ sở dữ liệu điều kiện đã thu hẹp đáng kể [1].

1.2.3 Mã giả thuật toán

Phần này trình bày mã giả đầy đủ của Relim, bám sát mô tả trong Section 2 của bài báo gốc [1]. Để tiện trình bày, thuật toán được chia thành ba thủ tục:

- RELIM: thủ tục chính, thực hiện tiền xử lý và khởi động đệ quy (tương ứng Mục 2.1 và mở đầu Mục 2.3 của bài báo gốc).
- RECURSIVE-RELIM: hàm đệ quy lõi – duyệt mảng danh sách, xuất tập phổ biến, gọi đệ quy và phân bổ lại (Mục 2.3).
- BUILD-CONDITIONAL-DB: thủ tục xây dựng cơ sở dữ liệu điều kiện cho một hạng mục (đoạn “second array of lists” trong Mục 2.3).

Các ký hiệu chính dùng trong mã giả được liệt kê ở Bảng 1.2 và nhất quán với Danh mục thuật ngữ ở đầu báo cáo.

Table 1.2: Quy ước ký hiệu trong mã giả

Ký hiệu	Ý nghĩa
$A[i]$	Phần tử thứ i của mảng các danh sách liên kết
$A[i].cnt$	Biến đếm hỗ trợ (số giao dịch chứa hạng mục i làm hạng mục dẫn đầu)
$A[i].head$	Con trỏ đầu danh sách liên kết
$e.next$	Con trỏ đến mắt xích kế tiếp trong danh sách
$e.trans$	Con trỏ vào phần còn lại của giao dịch (sau khi ẩn các hạng mục đứng trước)
P	Tiền tố – tập hạng mục đã xét trong các vòng đệ quy trước đó
B	Mảng danh sách của cơ sở dữ liệu điều kiện (conditional database)
σ	Ngưỡng hỗ trợ tối thiểu (minsup)

Thủ tục chính Relim

Algorithm 1 RELIM: thủ tục chính của thuật toán Relim

Require: Cơ sở dữ liệu giao dịch $\mathcal{D}$; ngưỡng tối thiểu σ .

Ensure: Tập $\mathcal{F}$ gồm mọi tập phổ biến của $\mathcal{D}$ kèm độ hỗ trợ.

```

1:  $\mathcal{F} \leftarrow \emptyset$  {tập kết quả}
2:  $freq \leftarrow \text{COUNTITEMFREQUENCIES}(\mathcal{D})$  {một lần quét,  $O(\sum_{T \in \mathcal{D}} |T|)$ }
3:  $\mathcal{I}^* \leftarrow \{i \in \mathcal{I} : freq[i] \geq \sigma\}$  {loại hạng mục không phổ biến}
4: Sắp xếp  $\mathcal{I}^*$  tăng dần theo  $freq$ , gán chỉ số  $1, \dots, k$  { $k = |\mathcal{I}^*|$ , định nghĩa thứ tự  $\preceq$ }
5: for all  $T \in \mathcal{D}$  do
6:   Loại các hạng mục  $\notin \mathcal{I}^*$  khỏi  $T$ 
7:   Sắp xếp các hạng mục còn lại của  $T$  theo thứ tự  $\preceq$ 
8: end for
9: Khởi tạo mảng  $A$  với  $k$  phần tử rỗng,  $A[i].cnt \leftarrow 0$ ,  $A[i].head \leftarrow \text{NIL}$ 
10: for all  $T \in \mathcal{D}$  với  $T \neq \emptyset$  do
11:    $i \leftarrow$  chỉ số hạng mục đầu tiên của  $T$ 
12:    $A[i].cnt \leftarrow A[i].cnt + 1$ 
13:   if  $|T| > 1$  then
14:     Cấp phát mắt xích mới  $e$ ;  $e.trans \leftarrow$  con trỏ vào vị trí thứ hai của  $T$  {ẩn hạng mục đầu bằng dịch con trỏ}
15:      $e.next \leftarrow A[i].head$ ;  $A[i].head \leftarrow e$ 
16:   end if {nếu  $|T| = 1$ : chỉ tăng counter, không cấp phát mắt xích (tối ưu Counter-Only)}
17: end for
18:  $\text{RECURSIVE-RELIM}(A, k, \emptyset, \sigma, \mathcal{F})$ 
19: return  $\mathcal{F}$ 
    
```

Diễn giải. Các dòng 2-4 thực hiện đúng một lần quét CSDL gốc để đếm tần suất và xác định tập hạng mục phổ biến đơn lẻ; thứ tự tăng dần được cố định ngay tại đây. Vòng lặp ở dòng 5-8 *thanh lọc* từng giao dịch: loại hạng mục không phổ biến và sắp xếp theo $\preceq$. Vòng lặp ở dòng 10-17 thực hiện *phân loại theo hạng mục đầu*: hạng mục đầu của mỗi giao dịch đã sắp xếp quyết định danh sách $A[i]$ mà mắt xích được chèn vào; bản thân hạng mục đầu được *ẩn ngầm* qua vị trí trong mảng (do đó **e.trans** trỏ tới hạng mục thứ

hai). Trường hợp giao dịch chỉ còn một hạng mục sau khi ẩn (thân giao dịch rỗng) được xử lý theo tối ưu Counter-Only của Borgelt: chỉ tăng biến đếm, không cấp phát mắt xích, vì không có “đuôi” nào để đẩy sâu hơn.

Hàm đệ quy Recursive-Relim

Algorithm 2 RECURSIVE-RELIM: hàm đệ quy lỗi

Require: Mảng danh sách A kích thước k ; tiền tố P ; ngưỡng σ ; tập kết quả $\mathcal{F}$ (tham chiếu).

```

1: for  $i \leftarrow 1$  to  $k$  do
2:   if  $A[i].cnt \geq \sigma$  then
3:      $\mathcal{F} \leftarrow \mathcal{F} \cup \{(P \cup \{i\}, A[i].cnt)\}$  {xuất tập phổ biến  $P \cup \{i\}$  với support}
4:      $B \leftarrow \text{BUILD-CONDITIONAL-DB}(A, i)$  {CSDL điều kiện cho  $i$ }
5:      $\text{RECURSIVE-RELIM}(B, i - 1, P \cup \{i\}, \sigma, \mathcal{F})$  {đệ quy với tiền tố mở rộng}
6:   end if
7:    $e \leftarrow A[i].head$ 
8:   while  $e \neq \text{NIL}$  do
9:      $e_{\text{next}} \leftarrow e.next$ 
10:    if  $e.trans$  vẫn còn ít nhất một hạng mục then
11:       $j \leftarrow$  chỉ số hạng mục đầu tiên của  $e.trans$ 
12:      Dịch  $e.trans$  sang hạng mục kế tiếp {ẩn  $j$ }
13:      if  $e.trans$  chưa rỗng then
14:         $e.next \leftarrow A[j].head$ ;  $A[j].head \leftarrow e$  {phân bổ lại sang  $A[j]$ }
15:      end if
16:       $A[j].cnt \leftarrow A[j].cnt + 1$ 
17:    end if
18:     $e \leftarrow e_{\text{next}}$ 
19:  end while
20:   $A[i].head \leftarrow \text{NIL}$ ;  $A[i].cnt \leftarrow 0$  {loại trừ  $i$  khỏi  $A$ }
21: end for

```

Diễn giải. Vòng **for** ngoài duyệt mảng A từ trái sang phải; biến chạy i là chỉ số của hạng mục đang đóng vai trò “hạng mục đầu của các tập phổ biến cần tìm”. Khối điều kiện ở dòng 2-6 hiện thực hai quan sát then chốt đã nêu ở Mục 1.2.1:

- *Dòng 3 - xuất tập phổ biến.* Nếu $A[i].cnt \geq \sigma$ thì $\{i\}$ phổ biến trong CSDL hiện tại, kết hợp với tiền tố P cho tập phổ biến $P \cup \{i\}$ ở cấp đệ quy này (Mệnh đề 1.2 đảm bảo nếu $A[i].cnt < \sigma$ thì mọi mở rộng đều không phổ biến — nhánh đệ quy được cắt tỉa).
- *Dòng 4-5 - gọi đệ quy.* CSDL điều kiện B tập hợp các giao dịch chứa i , đã loại i và các hạng mục đứng trước i . Mọi tập phổ biến của B mở rộng tiền tố $P \cup \{i\}$ cho ra tập phổ biến mới.

Vòng **while** ở dòng 8-19 thực hiện *phân bổ lại* (reassign): với mỗi mắt xích e trong $A[i]$, lấy hạng mục đầu tiên j hiện tại của $e.trans$ ($j > i$ vì danh sách đã sắp xếp tăng dần), ẩn j đi bằng một bước dịch con trỏ, rồi chèn e vào danh sách $A[j]$. Lưu ý cập nhật $A[j].cnt$ ngay cả khi mắt xích không được chèn (trường hợp $e.trans$ trở thành rỗng sau khi ẩn j) -

đây chính là tối ưu Counter-Only ở cấp độ quy này. Sau khi tất cả mắt xích đã rời $A[i]$, danh sách này được làm rỗng (dòng 20): hạng mục i được *loại trừ vĩnh viễn* khỏi CSDL hiện tại; mọi tập phổ biến tiếp theo sẽ có hạng mục đầu lớn hơn i .

Lưu ý kỹ thuật. Việc đệ quy ở dòng 5 chỉ truyền kích thước $i - 1$ (thay vì k): đây là tối ưu mà Borgelt [1, Section 2.4] đề xuất - trong CSDL điều kiện B , các vị trí $i, i + 1, \dots, k$ luôn rỗng (vì mọi giao dịch trong B ban đầu chứa i , và sau khi ẩn i , hạng mục đầu mới luôn nhỏ hơn i theo thứ tự $\preceq$). Việc này tránh các vòng lặp trống ở các cấp độ quy sâu.

Thủ tục Build-Conditional-DB

Algorithm 3 BUILD-CONDITIONAL-DB: dựng CSDL điều kiện cho hạng mục i

Require: Mảng danh sách A ; chỉ số i của hạng mục đang xét.

Ensure: Mảng danh sách B là CSDL điều kiện ứng với tiền tố $\{i\}$.

```

1: Khởi tạo mảng  $B$  với  $i - 1$  phần tử,  $B[j].cnt \leftarrow 0$ ,  $B[j].head \leftarrow \text{NIL}$  với mọi  $j < i$ 
2:  $e \leftarrow A[i].head$ 
3: while  $e \neq \text{NIL}$  do
4:   if  $e.trans$  vẫn còn ít nhất một hạng mục then
5:      $j \leftarrow$  chỉ số hạng mục đầu hiện tại của  $e.trans$ 
6:     Tạo bản sao mắt xích  $e' \leftarrow \text{COPY}(e)$ 
7:     Dịch  $e'.trans$  sang hạng mục kế tiếp  $\{\text{ẩn } j\}$ 
8:     if  $e'.trans$  chưa rỗng then
9:        $e'.next \leftarrow B[j].head$ ;  $B[j].head \leftarrow e'$ 
10:    end if
11:     $B[j].cnt \leftarrow B[j].cnt + 1$ 
12:  end if
13:   $e \leftarrow e.next$ 
14: end while
15: return  $B$ 

```

Diễn giải. Thủ tục này song hành với vòng phân bổ lại trong RECURSIVE-RELIM: cùng một mắt xích vừa được *phân bổ lại* (cập nhật vào A) vừa được *nhân bản* (chèn bản sao vào B). Điểm tinh tế là:

- Mắt xích trong B là *bản sao* của mắt xích trong A , không phải cùng một mắt xích. Cả hai bản sao đều trở vào *cùng một mảng giao dịch* (qua trường **trans**); việc dịch con trỏ chỉ thay đổi vị trí đọc trong mảng đó, không sao chép nội dung giao dịch. Đây là chìa khoá giúp Relim tiết kiệm bộ nhớ.
- Trường hợp $e.trans$ chỉ còn đúng một hạng mục: sau khi ẩn j , phần đuôi rỗng, nên không cấp phát mắt xích mới trong $B[j]$; chỉ tăng $B[j].cnt$ (tối ưu Counter-Only).
- Tối ưu cài đặt được Borgelt nêu trong Mục 2.4: nếu $A[i].cnt < \sigma$, có thể bỏ qua hoàn toàn bước BUILD-CONDITIONAL-DB vì không thể tìm thấy tập phổ biến nào trong nhánh đệ quy đó (điều này đã được kiểm tra ở dòng 2 của Thuật toán 2 trước khi gọi tới đây).

Mối liên hệ với cấu trúc dữ liệu. Trên hai cấu trúc song song A và B , mỗi vòng lặp ngoài của RECURSIVE-RELIM đồng thời (i) *thu hẹp* A bằng cách loại trừ hạng mục i và (ii) *nhân bản* một phần $A[i]$ sang B để đệ quy sâu hơn. Hai thao tác này tương ứng chính xác với hai cột trái và phải trong Hình 1 của bài báo Borgelt: cột trái là sự “tan rã” dần của mảng A qua bốn bước cho đến khi rỗng; cột phải là các mảng B được dựng tạm với tiền tố tương ứng (prefix e , prefix a , prefix c , prefix b).

1.2.4 Phân tích độ phức tạp

Bài báo gốc [1] chủ yếu đánh giá Relim qua thực nghiệm và không trình bày phân tích độ phức tạp hình thức. Trong phần này, chúng tôi suy ra các cận tiệm cận trực tiếp từ giả mã ở Mục 1.2.3. Quy ước các tham số sau, dùng thống nhất cho cả phân tích thời gian và không gian:

- $n = |\mathcal{D}|$: số giao dịch của CSDL gốc;
- $w = \frac{1}{n} \sum_{T \in \mathcal{D}} |T|$: chiều rộng giao dịch trung bình;
- $N = \sum_{T \in \mathcal{D}} |T| = n \cdot w$: tổng kích thước (số cặp *giao dịch-hạng mục*) - đại lượng then chốt trong các phân tích sau;
- $m = |\mathcal{I}|$: số hạng mục phân biệt; $k = |\mathcal{I}^*|$: số hạng mục phổ biến đơn lẻ (sau cắt tĩa, $k \leq m$);
- $|\mathcal{F}|$: số tập phổ biến đầu ra;
- σ : ngưỡng support tối thiểu.

Độ phức tạp thời gian

Chi phí tiền xử lý (cố định). Các bước trong Thuật toán 1:

- Đếm tần suất (dòng 2): một lần quét toàn bộ CSDL với $O(N)$ thao tác.
- Cắt tĩa và sắp xếp $\mathcal{I}^*$ (dòng 3-4): $O(m + m \log m)$.
- Sắp xếp các hạng mục trong mỗi giao dịch (dòng 5-8): mỗi giao dịch tốn $O(|T| \log |T|)$ nếu dùng so sánh; với $|T| \leq w_{\max}$, tổng chi phí tối đa $O(N \log w_{\max})$. Trên CSDL điển hình, đây không phải là điểm nghẽn.
- Khởi tạo mảng A và phân loại giao dịch theo hạng mục đầu (dòng 9-17): $O(n)$.

Tổng chi phí tiền xử lý: $\boxed{O(N \log w_{\max} + m \log m)}$. Khi $w_{\max}$ hằng số, đơn giản còn $O(N + m \log m)$.

Chi phí một cấp đệ quy. Xét một lời gọi RECURSIVE-RELIM($A, k', P, \sigma, \cdot$) ở một cấp đệ quy bất kỳ, với mảng A kích thước k' và tổng số mắt xích là L . Toàn bộ công việc ở cấp này gồm hai phần:

- (R1) Vòng phân bổ lại (dòng 8-19): mỗi mắt xích được *dịch con trở một lần* và chèn vào danh sách khác $O(1)$. Tổng chi phí cho cả mảng: $O(L + k')$.

(R2) Các lời gọi BUILD-CONDITIONAL-DB cho các nhánh phổ biến (dòng 4): mỗi mắt xích $e \in A[i]$ tạo *tối đa* một bản sao với chi phí $O(1)$. Tổng chi phí phần này, cộng dồn trên mọi i có $A[i].cnt \geq \sigma$, không vượt quá $O(L)$.

Vậy chi phí cục bộ (không tính đệ quy con) tại mỗi cấp là $O(L + k')$.

Tổng chi phí qua mọi cấp đệ quy. Quan sát then chốt [1, Section 2]: tổng số mắt xích sinh ra trên *toàn bộ* cây đệ quy bằng tổng kích thước các CSDL điều kiện. Dùng định nghĩa độ hỗ trợ, có thể viết:

$$\mathcal{W}_{\text{lists}} = \sum_{X \in \mathcal{F}} \text{sup}(X) \leq |\mathcal{F}| \cdot n. \quad (1.13)$$

Trực giác: với mỗi tập phổ biến $X = \{i_1 \prec i_2 \prec \dots \prec i_\ell\}$ được xuất ra, mỗi giao dịch $T \supseteq X$ đóng góp đúng một mắt xích vào CSDL điều kiện ở cấp đệ quy ứng với tiền tố $\{i_1, \dots, i_{\ell-1}\}$, danh sách $A[i_\ell]$. Cộng dồn cho tất cả tập phổ biến cho cận $\sum_X \text{sup}(X)$. Mỗi mắt xích lại được xử lý trong $O(1)$ ở cấp đệ quy của nó, nên tổng công việc xử lý mắt xích là $O(\sum_X \text{sup}(X))$.

Kèm theo chi phí duyệt mảng (mỗi cấp đệ quy có thêm $O(k')$ phép duyệt trống), ta có ước lượng tổng quát:

$$T_{\text{RELIM}}(\mathcal{D}, \sigma) = O\left(N + m \log m + \sum_{X \in \mathcal{F}} \text{sup}(X)\right). \quad (1.14)$$

Trường hợp tốt nhất. Khi σ rất cao, hầu hết hạng mục không phổ biến: $k \rightarrow 0$ và $|\mathcal{F}|$ nhỏ. Trong cực đoan $k = 0$ (không có hạng mục nào phổ biến), thuật toán dừng ngay sau khi đếm tần suất:

$$T_{\text{best}} = O(N + m \log m). \quad (1.15)$$

Ngay cả khi $k > 0$ nhưng vẫn nhỏ, vòng đệ quy thừa thớt và thuật toán xấp xỉ tuyến tính theo N .

Trường hợp xấu nhất. Khi $\sigma = 1$ (mọi tập con khác rỗng đều phổ biến), số tập phổ biến đạt $|\mathcal{F}| = 2^k - 1$ và $\sum_X \text{sup}(X)$ có thể đạt $O(n \cdot 2^k)$. Thay vào (1.14):

$$T_{\text{worst}} = O(n \cdot 2^k). \quad (1.16)$$

Đây là cận xấu nhất chung cho *mọi* thuật toán FIM khai phá đầy đủ tập phổ biến: số đầu ra đã có thể là 2^k . Sự bùng nổ này không phải do thiết kế của Relim mà do kích thước bài toán.

Trường hợp trung bình. Khó đưa cận khít kín do phụ thuộc phân phối dữ liệu. Borgelt báo cáo thực nghiệm trên năm bộ dữ liệu chuẩn (BMS-Webview-1, T10I4D100K, census, chess, mushroom) cho thấy [1, Section 3]:

- Đường cong $\log_{10}$ (thời gian) của Relim luôn nhanh hơn Apriori, đôi khi nhanh hơn Eclat;
- FP-Growth thường nhanh hơn Relim trên dữ liệu dày (**chess**, **mushroom**, **census**), do hưởng lợi từ chia sẻ tiền tố trong FP-tree mà Relim không có;
- Khoảng cách giữa Relim và Apriori giãn rộng khi σ giảm.

Có thể tóm gọn: trên dữ liệu thưa, $|\mathcal{F}|$ vừa phải và $\sum_X \text{sup}(X)$ tăng tuyến tính theo $1/\sigma$; thực nghiệm phù hợp với cận lý thuyết (1.14).

Các yếu tố ảnh hưởng chính đến hiệu năng. Từ phân tích trên có thể nêu rõ:

- **Ngưỡng σ :** ảnh hưởng *lớn nhất* thông qua kích thước $|\mathcal{F}|$. Quan hệ $|\mathcal{F}|$ với σ thường là siêu tuyến tính (tăng đột biến khi σ giảm dưới một ngưỡng nào đó).
- **Kích thước CSDL n :** ảnh hưởng *tuyến tính* đến cả pha tiền xử lý ($N = nw$) và pha đệ quy ($\sum_X \sup(X)$ tỉ lệ với n khi cố định mật độ).
- **Chiều rộng giao dịch trung bình w :** tăng w làm N tăng, đồng thời tăng kích thước các CSDL điều kiện (giao dịch dài hơn $\Rightarrow$ nhiều mắt xích sống sót qua nhiều tầng đệ quy). Đây là yếu tố nhạy với Relim hơn so với FP-Growth (do FP-tree chia sẻ tiền tố).
- **Số hạng mục phổ biến k :** quyết định độ sâu đệ quy tối đa và cận trên xấu nhất 2^k .
- **Mật độ dữ liệu (dense vs sparse):** dữ liệu dày sinh nhiều tập phổ biến hơn cùng σ ; thực nghiệm xác nhận Relim chậm hơn FP-Growth trên dữ liệu dày như chess hay mushroom [1, Figures 5-6].

Độ phức tạp không gian

Bộ nhớ tĩnh. CSDL gốc lưu dưới dạng n mảng số nguyên có tổng kích thước N phần tử, cộng mảng A kích thước k . Bộ nhớ cố định: $O(N + k)$. Mỗi giao dịch chỉ được lưu *một lần*; các bước đệ quy sau không sao chép nội dung giao dịch (chỉ dịch con trỏ vào mảng đã có), khác hẳn FP-Growth cần dựng cây mới ở mỗi cấp.

Bộ nhớ động cho mắt xích danh sách. Đây là điểm thú vị nhất trong phân tích bộ nhớ. Gọi d là độ sâu đệ quy tối đa ($d \leq k$). Khi gọi đệ quy ở dòng 5, mảng B ở cấp con *cùng tồn tại* với mảng A ở cấp cha trong thời gian chạy nhánh con. Tuy nhiên, do thiết kế phân loại theo hạng mục đầu:

$$\sum_{i=1}^k \#\{\text{mắt xích trong } A[i]\} \leq n, \quad (1.17)$$

nên *tổng số mắt xích sống đồng thời ở mỗi cấp đệ quy không vượt quá n* . Sau cấp đệ quy, một số giao dịch bị loại bỏ (do thân trở thành rỗng), nên ở các cấp đệ quy con, số mắt xích thường nhỏ hơn n đáng kể.

Cận trên không gian (chưa tối ưu):

$$S_{\text{naive}} = O(N + d \cdot n). \quad (1.18)$$

Borgelt [1, Section 2.4] đề xuất tối ưu *cấp phát theo khối (block allocation)*: cấp phát n mắt xích cho mỗi cấp đệ quy trong một mảng liên kề, và lưu các mảng này ở một vùng nhớ toàn cục để *tái sử dụng* khi đệ quy quay lui. Cận trên vẫn là $O(d \cdot n)$, nhưng (i) tránh phân mảnh, (ii) không phải gọi `malloc/new` ở mỗi cấp đệ quy - một thắng lợi lớn về hằng số trong thực tiễn.

Trường hợp tốt nhất, trung bình, xấu nhất.

- *Tốt nhất*: $d = 0$ (không có hạng mục phổ biến) hoặc $d = O(\log n)$ trên dữ liệu phân tán đều: $S = O(N)$.
- *Trung bình*: d tỉ lệ với chiều dài tập phổ biến dài nhất. Trên dữ liệu thực, d thường nhỏ hơn k đáng kể: $S = O(N + d \cdot n)$ với $d \ll k$.
- *Xấu nhất*: $d = k$ khi σ rất thấp: $S = O(N + k \cdot n) = O(N + kn)$.

So sánh không gian với các thuật toán đương thời. Apriori cần lưu các tập ứng viên ở mỗi cấp k , có thể vượt xa $|\mathcal{F}|$. FP-Growth lưu FP-tree gốc (chia sẻ tiền tố nên tốt trên dày, kém trên thưa) cộng các cây điều kiện trên đường đệ quy - chi phí khởi tạo và tái cấu trúc cây có hằng số lớn. Relim đặt giữa: không có lợi thế chia sẻ tiền tố như FP-Growth, nhưng cấu trúc danh sách liên kết đơn cực kỳ nhẹ (mỗi mắt xích chỉ 2 con trỏ, tức 8 byte trên máy 32-bit [1, Section 2.4]); kết hợp với tối ưu Counter-Only và block allocation, Relim thường tiết kiệm bộ nhớ hơn FP-Growth trên dữ liệu thưa.

1.2.5 Vị trí của Relim trong dòng phát triển FIM

Để đánh giá đúng đóng góp của Relim, cần đặt nó trong bức tranh tổng thể của lĩnh vực khai phá tập phổ biến từ những năm 1990 đến đầu những năm 2000.

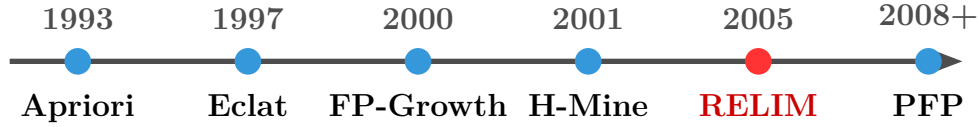


Figure 1.4: Dòng thời gian phát triển của các thuật toán Khai phá tập phổ biến tiêu biểu.

1.2.5.1 Thế hệ thứ nhất: Sinh tập ứng viên theo chiều rộng

Apriori (Agrawal và Swami, 1993 [2]) đặt nền móng cho FIM bằng phương pháp sinh tập ứng viên theo tầng (level-wise). Thuật toán này cần k lần quét cơ sở dữ liệu và số ứng viên tăng theo hàm mũ khi σ giảm.

Eclat (Zaki, 1997 [6]) chuyển sang biểu diễn dọc (vertical format): mỗi hạng mục i được gán tập định danh TID(i); support tính bằng phép giao: $\text{sup}(X) = |\bigcap_{i \in X} \text{TID}(i)|$. Khi dữ liệu dày, TID-set lớn khiến phép giao tốn kém.

1.2.5.2 Thế hệ thứ hai: Thay thế việc sinh ứng viên bằng đệ quy

FP-Growth (Han, Pei và Yin, 2000 [3]) nén $\mathcal{D}$ vào FP-tree (hai lần quét), khai phá đệ quy các cây điều kiện mà không sinh ứng viên. Rất mạnh trên dữ liệu dày nhờ chia sẻ tiền tố, nhưng việc xây dựng và tái cấu trúc FP-tree đòi hỏi cài đặt phức tạp.

H-Mine (Pei et al., 2001 [4]) dùng siêu liên kết (hyperlinks) thay cây tương minh, đơn giản hơn FP-Growth về cài đặt nhưng vẫn cần quản lý mạng con trỏ phức tạp qua nhiều tầng.

1.2.5.3 Đóng góp của Relim (Borgelt, 2005)

Borgelt [1] đề xuất Relim với triết lý *tối giản hóa*: kế thừa nguyên lý đệ quy loại trừ nhưng loại bỏ hoàn toàn cây và siêu liên kết. Toàn bộ thuật toán hoạt động trên một mảng danh sách liên kết đơn và một hàm đệ quy duy nhất với số dòng mã rất nhỏ.

1.2.5.4 Các hướng tiếp cận cùng thời

Đánh đổi cho sự đơn giản là Relim không khai thác được cơ chế chia sẻ tiền tố của FP-tree. Trên dữ liệu dày (**chess**, **mushroom**), FP-Growth vượt xa Relim vì có thể biểu diễn nhiều giao dịch bằng một nhánh dùng chung.

Các hướng tiếp cận khác cùng giai đoạn gồm: (i) **dEclat** (Zaki và Gouda, 2003 [7]) thay TID-set bằng *diffset* để tiết kiệm bộ nhớ trên dữ liệu dày; (ii) **LCM** (Uno et al., 2004 [8]) khai phá qua các tập đóng, đạt hiệu năng xuất sắc trong các cuộc thi chuẩn FIM.

1.2.5.5 Các hướng phát triển sau Relim

Sau Relim, nghiên cứu FIM tiếp tục tập trung vào ba hướng chính: (i) tối ưu biểu diễn dữ liệu, (ii) khai phá tập đóng/tối đại, và (iii) khai phá song song trên dữ liệu lớn.

AFOPT (Liu et al., 2006 [10]) cải tiến FP-Growth bằng cấu trúc *Ascending Frequency Ordered Prefix Tree*, giúp giảm chi phí xây dựng conditional tree và tăng hiệu quả trên dữ liệu dày.

Các thuật toán khai phá tập đóng như **Charm** (Zaki và Hsiao, 2002 [9]) và các biến thể của **LCM** [8] tiếp tục được tối ưu nhằm giảm số lượng mẫu cần sinh, đặc biệt hiệu quả khi số tập phổ biến rất lớn.

Trong giai đoạn dữ liệu lớn (*big data*), các thuật toán như **Parallel FP-Growth (PFP)** (Li et al., 2008 [11]) trên Hadoop/Spark và các phương pháp GPU-based FIM được phát triển nhằm mở rộng khả năng xử lý lên hàng tỷ giao dịch.

Gần đây, nghiên cứu FIM còn mở rộng sang: (i) khai phá luồng dữ liệu (*stream mining*), (ii) khai phá tập hữu ích cao (*high-utility itemset mining*), và (iii) khai phá trên dữ liệu không chắc chắn (*uncertain data mining*).

Bảng 1.3 tóm tắt vị trí của Relim trong dòng phát triển này.

Table 1.3: So sánh các thuật toán FIM qua các thế hệ

Thuật toán	Năm	Cấu trúc DL	Ưu điểm	Hạn chế
Apriori	1993	Horizontal	Đơn giản, nền tảng lý thuyết	Nhiều lần quét, sinh nhiều ứng viên
Eclat	1997	TID-set dọc	Một lần đọc; tốt trên thưa	TID-set lớn khi dense
FP-Growth	2000	FP-tree	Rất nhanh trên dense	Cài đặt phức tạp, tốn RAM
H-Mine	2001	H-struct	Nhanh hơn Apriori	Con trỏ phức tạp nhiều tầng
dEclat	2003	Diffset dọc	Tiết kiệm RAM trên dense	Phức tạp hơn Eclat
LCM	2004	Closed sets	Hiệu năng tốt nhất thực tế	Phức tạp về lý thuyết
Relim	2005	Mảng linked list	Cực kỳ đơn giản; ít RAM; tốt trên thưa	Chậm hơn FP-Growth trên dense
AFOPT	2006	Prefix tree tối ưu	Tăng tốc FP-Growth	Cấu trúc vẫn phức tạp
PFP	2008	Distributed FP-tree	Xử lý dữ liệu lớn song song	Overhead phân tán

Chương 2

VÍ DỤ MINH HỌA TAY

Chương này trình bày hai ví dụ thực hiện tay hoàn toàn để làm rõ cơ chế hoạt động của Relim. **Ví dụ 2.1** minh họa luồng xử lý tổng quát. **Ví dụ 2.2** tập trung vào tình huống đặc biệt *Counter-Only*, một tối ưu hóa đặc trưng của Relim.

2.1 Ví dụ 1: Minh họa tổng quát

2.1.1 Thiết lập

$$\mathcal{I} = \{a, b, c, d, e, f\} \quad |\mathcal{D}| = 6 \quad \sigma = 3$$

Table 2.1: Cơ sở dữ liệu gốc (Ví dụ 1)

TID	Giao dịch
T_1	$\{a, b, c, d\}$
T_2	$\{b, c, e, f\}$
T_3	$\{a, b, d\}$
T_4	$\{b, c, d, e\}$
T_5	$\{a, c, d\}$
T_6	$\{b, d, e\}$

2.1.2 Bước tiền xử lý

Table 2.2: Tần suất các hạng mục và thứ tự sắp xếp

Hạng mục	f	a	e	c	b	d
Tần suất	1	3	3	4	5	5
Thứ tự	–	1	2	3	4	5
Phổ biến?	×	✓	✓	✓	✓	✓

Đếm tần suất. Hạng mục f có tần suất $1 < \sigma = 3$ nên bị **loại bỏ** khỏi cơ sở dữ liệu. Các hạng mục khác được giữ nguyên. Thứ tự sắp xếp tần suất tăng dần (tie-break theo bảng chữ cái):

$$\underbrace{a}_3 < \underbrace{e}_3 < \underbrace{c}_4 < \underbrace{b}_5 \approx \underbrace{d}_5.$$

Cơ sở dữ liệu sau tiền xử lý được thể hiện trong Bảng 2.3.

Table 2.3: Cơ sở dữ liệu sau sắp xếp ($a < e < c < b < d$)

TID	Giao dịch (đã sắp xếp)	Hạng mục đứng đầu
T_1	a, c, b, d	a
T_2	e, c, b	e
T_3	a, b, d	a
T_4	e, c, b, d	e
T_5	a, c, d	a
T_6	e, b, d	e

Trạng thái mảng A sau khởi tạo được thể hiện trong Bảng 2.4.

Table 2.4: Mảng A sau khởi tạo - Ví dụ 1

Hạng mục	cnt	Danh sách (hậu tố)	Nguồn
a	3	$(c, b, d) \rightarrow (b, d) \rightarrow (c, d)$	T_1, T_3, T_5
e	3	$(c, b) \rightarrow (c, b, d) \rightarrow (b, d)$	T_2, T_4, T_6
c	0	<i>rỗng</i>	-
b	0	<i>rỗng</i>	-
d	0	<i>rỗng</i>	-

2.1.3 Thực hiện thuật toán từng bước

Bước 1 - Xử lý hạng mục a

$$\sup(a) = 3 \geq \sigma$$

Ghi nhận: $\{a\} \in \mathcal{F}$ với $\text{support} = 3$.

Xây dựng DB_a từ hậu tố các giao dịch trong $A[a]$:

Giao dịch gốc	Hậu tố (sau khi ẩn a)	Đưa vào DB_a
$T_1 : (a, c, b, d)$	(c, b, d)	$j=c$; còn $(b, d) \rightarrow$ mất xích $\rightarrow B_a[c]$; $B_a[c].cnt += 1$
$T_3 : (a, b, d)$	(b, d)	$j=b$; còn $(d) \rightarrow$ mất xích $\rightarrow B_a[b]$; $B_a[b].cnt += 1$
$T_5 : (a, c, d)$	(c, d)	$j=c$; còn $(d) \rightarrow$ mất xích $\rightarrow B_a[c]$; $B_a[c].cnt += 1$

Trạng thái DB_a sau xây dựng: $B_a[c].cnt = 2$ (danh sách: $(b, d), (d)$),
 $B_a[b].cnt = 1$ (danh sách: (d)), $B_a[d].cnt = 0$ (rỗng).

Đệ quy trên DB_a :

- Xử lý c trong DB_a : $B_a[c].cnt = 2 < \sigma \Rightarrow$ bỏ qua. Phân bổ lại từ $B_a[c]$:
 $(b, d) \rightarrow B_a[b]$, $B_a[b].cnt += 1$; $(d) \rightarrow$ Counter-Only: $B_a[d].cnt += 1$.
- Xử lý b trong DB_a : $B_a[b].cnt = 2 < \sigma \Rightarrow$ bỏ qua. Phân bổ lại từ $B_a[b]$:
 $(d) \rightarrow$ Counter-Only $B_a[d].cnt += 1$; $(d) \rightarrow$ Counter-Only $B_a[d].cnt += 1$.
 Tổng $B_a[d].cnt = 3$.
- Xử lý d trong DB_a : $B_a[d].cnt = 3 \geq \sigma \Rightarrow$ **ghi nhận** $\{a, d\} \in \mathcal{F}$ (support = 3).

Phân bổ lại (reassign) từ $A[a]$ về mảng A : Mỗi mất xích trong $A[a]$ được chuyển sang danh sách theo hạng mục dẫn đầu của nó: $(c, b, d) \rightarrow A[c]$,
 $A[c].cnt += 1$; $(b, d) \rightarrow A[b]$, $A[b].cnt += 1$; $(c, d) \rightarrow A[c]$, $A[c].cnt += 1$.
 Cập nhật tổng: $A[c].cnt += 2$, $A[b].cnt += 1$.

Table 2.5: Trạng thái mảng A sau Bước 1 (sau reassign a)

Hạng mục	cnt	Danh sách (hậu tố)
a	0 (đã xử lý)	rỗng
e	3	$(c, b) \rightarrow (c, b, d) \rightarrow (b, d)$
c	2	(b, d) [từ T_1] $\rightarrow (d)$ [từ T_5]
b	1	(d) [từ T_3]
d	0	rỗng

Bước 2 - Xử lý hạng mục e

$$\text{sup}(e) = 3 \geq \sigma$$

Ghi nhận: $\{e\} \in \mathcal{F}$ với $\text{support} = 3$.

Xây dựng DB_e từ hậu tố trong $A[e]$:

Giao dịch gốc	Hậu tố (ẩn e)	Đưa vào DB_e
$T_2 : (e, c, b)$	(c, b)	Mắt xích $\rightarrow B_e[c]$
$T_4 : (e, c, b, d)$	(c, b, d)	Mắt xích $\rightarrow B_e[c]$
$T_6 : (e, b, d)$	(b, d)	Mắt xích $\rightarrow B_e[b]$

Đếm trong DB_e : $\text{sup}_{DB_e}(c) = 2$, $\text{sup}_{DB_e}(b) = 3$, $\text{sup}_{DB_e}(d) = 2$.

Chỉ b đạt ngưỡng. Độ quy trên DB_e :

- c : $\text{sup} = 2 < \sigma \Rightarrow$ bỏ qua.
- b : $\text{sup} = 3 \geq \sigma \Rightarrow$ **ghi nhận** $\{e, b\} \in \mathcal{F}$ ($\text{support} = 3$). Độ quy sâu hơn: $\text{sup}_{DB_{eb}}(d) = 2 < \sigma \Rightarrow$ dừng.

Phân bổ lại từ $A[e]$ về mảng A : $(c, b) \rightarrow A[c]$, $A[c].\text{cnt} += 1$; $(c, b, d) \rightarrow A[c]$, $A[c].\text{cnt} += 1$; $(b, d) \rightarrow A[b]$, $A[b].\text{cnt} += 1$.

Cập nhật tổng: $A[c].\text{cnt} += 2$, $A[b].\text{cnt} += 1$.

Table 2.6: Trạng thái mảng A sau Bước 2 (sau reassign e)

Hạng mục	cnt	Danh sách (hậu tố)
a	0 (đã xử lý)	rỗng
e	0 (đã xử lý)	rỗng
c	4	$(b, d) \rightarrow (d)$ [từ a] + $(b) \rightarrow (b, d)$ [từ e]
b	2	(d) [từ a] + (d) [từ e]
d	0	rỗng

Bước 3 - Xử lý hạng mục c

$$\sup(c) = 4 \geq \sigma$$

Ghi nhận: $\{c\} \in \mathcal{F}$ với $\text{support} = 4$.

Xây dựng DB_c : $A[c]$ sau bước reassign của a và e chứa hậu tố từ T_1, T_2, T_4, T_5 .

Nguồn	Hậu tố trong $A[c]$	Khi ẩn $c \rightarrow$ đưa vào DB_c
T_1 (từ a)	(c, b, d)	$(b, d) \rightarrow$ mắt xích $\rightarrow B_c[b]$
T_5 (từ a)	(c, d)	$(d) \rightarrow$ Counter-Only: $B_c[d].cnt += 1$
T_2 (từ e)	(c, b)	$(b) \rightarrow$ Counter-Only: $B_c[b].cnt += 1$
T_4 (từ e)	(c, b, d)	$(b, d) \rightarrow$ mắt xích $\rightarrow B_c[b]$

Đếm trong DB_c : $\sup_{DB_c}(b) = 3$ (T_1, T_2, T_4), $\sup_{DB_c}(d) = 3$ (T_1, T_4, T_5).

Cả b và d đều $\geq \sigma$. Đệ quy trên DB_c :

- b : $\sup = 3 \geq \sigma \Rightarrow$ **ghi nhận** $\{c, b\} \in \mathcal{F}$ ($\text{support} = 3$).
Đệ quy DB_{cb} : hậu tố (d) và $(d) \rightarrow \sup_{DB_{cb}}(d) = 2 < \sigma \Rightarrow$ dừng.
- d : $\sup = 3 \geq \sigma \Rightarrow$ **ghi nhận** $\{c, d\} \in \mathcal{F}$ ($\text{support} = 3$).

Phân bổ lại từ $A[c]$ về mảng A : Mỗi mắt xích trong $A[c]$ được chuyển theo hạng mục dẫn đầu: $(b, d) \rightarrow A[b]$, $A[b].cnt += 1$; $(d) \rightarrow$ Counter-Only: $A[d].cnt += 1$; $(b) \rightarrow$ Counter-Only: $A[b].cnt += 1$; $(b, d) \rightarrow A[b]$, $A[b].cnt += 1$.

Cập nhật tổng: $A[b].cnt += 3$, $A[d].cnt += 1$.

Table 2.7: Trạng thái mảng A sau Bước 3 (sau reassign c)

Hạng mục	cnt	Danh sách (hậu tố)
a	0 (đã xử lý)	rỗng
e	0 (đã xử lý)	rỗng
c	0 (đã xử lý)	rỗng
b	5	(d) [từ a] + (d) [từ e] + (d) [từ T_1 qua c] + (d) [từ T_4 qua c] + [1 counter-only T_2]
d	1	rỗng (counter-only từ T_5 qua c)

Bước 4 - Xử lý hạng mục b

$$\text{sup}(b) = 5 \geq \sigma$$

Ghi nhận: $\{b\} \in \mathcal{F}$ với $\text{support} = 5$.

Nguồn gốc $A[b].cnt = 5$:

- Bước 1 (reassign a): T_3 có hậu tố $(b, d) \rightarrow A[b].cnt += 1$, mất xích (d) .
- Bước 2 (reassign e): T_6 có hậu tố $(b, d) \rightarrow A[b].cnt += 1$, mất xích (d) .
- Bước 3 (reassign c): T_1 hậu tố $(b, d) \rightarrow$ mất xích (d) ; T_4 hậu tố $(b, d) \rightarrow$ mất xích (d) ; T_2 hậu tố $(b) \rightarrow$ **Counter-Only**. Tổng: $A[b].cnt += 3$.

Xây dựng DB_b :

Nguồn	Hậu tố trong $A[b]$	Khi ẩn $b \rightarrow$ đưa vào DB_b
T_3 (từ a)	(d)	Counter-Only: $B_b[d].cnt += 1$
T_6 (từ e)	(d)	Counter-Only: $B_b[d].cnt += 1$
T_1 (từ c)	(d)	Counter-Only: $B_b[d].cnt += 1$
T_4 (từ c)	(d)	Counter-Only: $B_b[d].cnt += 1$

Đếm trong DB_b : $\text{sup}_{DB_b}(d) = 4$ (từ T_1, T_3, T_4, T_6 ; tất cả đều *Counter-Only* vì hậu tố chỉ còn 1 item).

d : $\text{sup} = 4 \geq \sigma \Rightarrow$ **ghi nhận** $\{b, d\} \in \mathcal{F}$ ($\text{support} = 4$).

Đệ quy DB_{bd} : rỗng (mọi hậu tố đều *Counter-Only*, không còn mất xích nào) $\Rightarrow$ dừng.

Phân bổ lại: 4 mất xích (d) chuyển sang $A[d]$. Cập nhật: $A[d].cnt += 4$.

Table 2.8: Trạng thái mảng A sau Bước 4 (sau reassign b)

Hạng mục	cnt	Danh sách
a	0 (đã xử lý)	rỗng
e	0 (đã xử lý)	rỗng
c	0 (đã xử lý)	rỗng
b	0 (đã xử lý)	rỗng
d	5	rỗng (<i>counter-only tích lũy</i>)

Bước 5 - Xử lý hạng mục d

$$\text{sup}(d) = 5 \geq \sigma$$

Ghi nhận: $\{d\} \in \mathcal{F}$ với $\text{support} = 5$.

Nguồn gốc $A[d].cnt = 5$ (tích lũy qua Counter-Only và reassign dây chuyền):

- Bước 3 (reassign c): (d) từ T_5 (qua c) $\rightarrow$ Counter-Only $A[d].cnt += 1$.
- Bước 4 (reassign b): 4 mắt xích (d) từ T_1, T_3 (qua $a \rightarrow c \rightarrow b$), T_4, T_6 (qua $e \rightarrow c/b$) $\rightarrow$ Counter-Only $A[d].cnt += 4$.
- Tổng tích lũy $= 1 + 4 = 5$.

d là hạng mục cuối cùng trong thứ tự, không có hạng mục nào đứng sau d . Danh sách $A[d]$ không có mắt xích (toàn bộ đều Counter-Only) nên không xây dựng DB_d và không có đệ quy. Thuật toán **kết thúc**.

2.1.4 Tập kết quả

$$\mathcal{F} = \{\{a\}:3, \{e\}:3, \{c\}:4, \{b\}:5, \{d\}:5, \{a,d\}:3, \{e,b\}:3, \{c,b\}:3, \{c,d\}:3, \{b,d\}:4\}$$

2.1.5 Cây đệ quy (Recursion Tree)

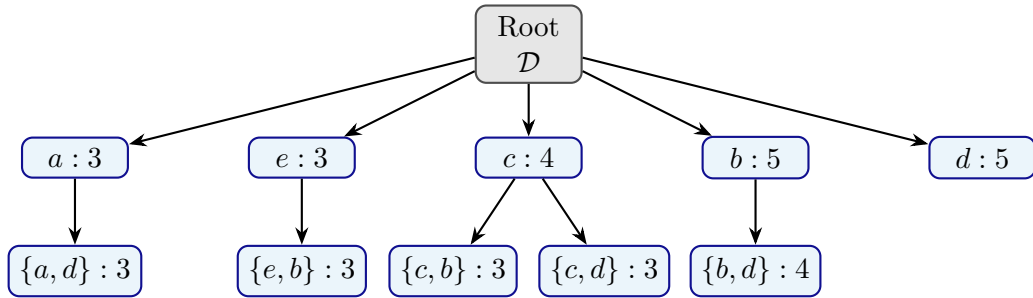


Figure 2.1: Cây đệ quy của thuật toán RELIM trên Ví dụ 1. Các nhánh con có $\text{support} < \sigma$ bị cắt tỉa nên không hiển thị.

2.1.6 Kiểm tra chéo

 Table 2.9: Kiểm tra chéo toàn bộ tập hạng mục (Ví dụ 1, $\sigma = 3$)

Tập	sup	Kết luận	Tập	sup	Kết luận
1-tập (6 tập, gồm cả f đã loại ở tiền xử lý):					
$\{a\}$	3	Phổ biến	$\{b\}$	5	Phổ biến
$\{e\}$	3	Phổ biến	$\{d\}$	5	Phổ biến
$\{c\}$	4	Phổ biến	$\{f\}$	1	Loại (tiền xử lý)
2-tập ($\binom{5}{2} = 10$ tập):					
$\{a, e\}$	0	Loại	$\{e, b\}$	3	Phổ biến
$\{a, c\}$	2	Loại	$\{e, d\}$	2	Loại
$\{a, b\}$	2	Loại	$\{c, b\}$	3	Phổ biến
$\{a, d\}$	3	Phổ biến	$\{c, d\}$	3	Phổ biến
$\{e, c\}$	2	Loại	$\{b, d\}$	4	Phổ biến
3-tập ($\binom{5}{3} = 10$ tập):					
$\{a, e, c\}$	0	Loại	$\{a, c, d\}$	2	Loại
$\{a, e, b\}$	0	Loại	$\{a, b, d\}$	2	Loại
$\{a, e, d\}$	0	Loại	$\{e, c, b\}$	2	Loại
$\{a, c, b\}$	1	Loại	$\{e, c, d\}$	1	Loại
$\{e, b, d\}$	2	Loại	$\{c, b, d\}$	2	Loại
4-tập ($\binom{5}{4} = 5$ tập):					
$\{a, e, c, b\}$	0	Loại	$\{a, c, b, d\}$	1	Loại
$\{a, e, c, d\}$	0	Loại	$\{e, c, b, d\}$	1	Loại
$\{a, e, b, d\}$	0	Loại			
5-tập ($\binom{5}{5} = 1$ tập):					
$\{a, e, c, b, d\}$	0	Loại			

Kết quả kiểm tra chéo xác nhận $\mathcal{F}$ do Relim tìm được là đầy đủ và chính xác.

2.2 Ví dụ 2: Tình huống đặc biệt - Counter-Only Transactions

2.2.1 Mô tả tình huống đặc biệt

Tình huống Counter-Only xảy ra khi trong quá trình xây dựng cơ sở dữ liệu điều kiện (BUILD-CONDITIONAL-DB) hoặc phân bổ lại (REASSIGN), một giao dịch sau khi ẩn hạng mục dẫn đầu chỉ còn lại *đúng một* hạng mục duy nhất. Thay vì cấp phát mắt xích mới (tồn bộ nhớ và thời gian), Relim chỉ tăng biến đếm $A[j].cnt$. Ví dụ này được thiết kế với nhiều giao dịch ngắn (2 item) để Counter-Only xuất hiện *tự nhiên và liên tục* trong các bước đệ quy, đặc biệt tại Bước 3 nơi **toàn bộ** thông tin được truyền qua biến đếm mà không cấp phát bất kỳ mắt xích nào.

2.2.2 Thiết lập

$$\mathcal{I} = \{a, b, c, d\} \quad |\mathcal{D}| = 5 \quad \sigma = 3$$

Table 2.10: Cơ sở dữ liệu gốc và sau tiền xử lý (Ví dụ 2)

TID	Giao dịch gốc	Sau tiền xử lý ($a < c < d < b$)
T_1	$\{a, b, c, d\}$	a, c, d, b
T_2	$\{a, b\}$	a, b
T_3	$\{a, c\}$	a, c
T_4	$\{b, c, d\}$	c, d, b
T_5	$\{b, d\}$	d, b

Table 2.11: Tần suất và thứ tự sắp xếp (Ví dụ 2)

Hạng mục	a	c	d	b
Tần suất	3	3	3	4
Thứ tự	1	2	3	4

2.2.3 Xây dựng mảng ban đầu

Theo Thuật toán 1, mỗi giao dịch có $|T| > 1$ luôn tạo mắt xích trong danh sách $A[i]$ (với i là hạng mục đứng đầu). Counter-Only chỉ được áp dụng bên trong BUILD-CONDITIONAL-DB và REASSIGN, không áp dụng tại khởi tạo.

Table 2.12: Phân loại giao dịch vào mảng A theo Thuật toán 1

TID	H.mục đầu	Hậu tố còn lại	$ T $	Hành động
T_1	a	(c, d, b)	4	$A[a].cnt += 1$; tạo mắt xích $\rightarrow A[a]$
T_2	a	(b)	2	$A[a].cnt += 1$; tạo mắt xích $\rightarrow A[a]$
T_3	a	(c)	2	$A[a].cnt += 1$; tạo mắt xích $\rightarrow A[a]$
T_4	c	(d, b)	3	$A[c].cnt += 1$; tạo mắt xích $\rightarrow A[c]$
T_5	d	(b)	2	$A[d].cnt += 1$; tạo mắt xích $\rightarrow A[d]$

Table 2.13: Trạng thái mảng A sau khởi tạo (Ví dụ 2)

H.mục	cnt	Danh sách liên kết	Ghi chú
a	3	$(c, d, b) \rightarrow (b) \rightarrow (c)$	3 mắt xích từ T_1, T_2, T_3
c	1	(d, b)	1 mắt xích từ T_4
d	1	(b)	1 mắt xích từ T_5
b	0	<i>rỗng</i>	Không có giao dịch nào có b đứng đầu

2.2.4 Thực hiện thuật toán từng bước

Bước 1 - Xử lý hạng mục a

$$\sup(a) = 3 \geq \sigma$$

Ghi nhận: $\{a\} \in \mathcal{F}$ với $\text{support} = 3$.

Xây dựng DB_a từ 3 mắt xích trong $A[a]$:

Nguồn	Hậu tố	Ẩn h.mục đầu	Đưa vào DB_a
T_1	(c, d, b)	$j=c$; còn (d, b)	Mắt xích $\rightarrow B_a[c]$; $B_a[c].cnt += 1$
T_2	(b)	$j=b$; còn rỗng	Counter-Only: $B_a[b].cnt += 1$
T_3	(c)	$j=c$; còn rỗng	Counter-Only: $B_a[c].cnt += 1$

Đếm trong DB_a : $\sup_{DB_a}(c) = 2$ (T_1 mắt xích + T_3 Counter-Only), $\sup_{DB_a}(b) = 1$ (T_2 Counter-Only).

Không có hạng mục nào đạt $\sigma = 3 \Rightarrow$ **không** sinh thêm tập với tiền tố $\{a\}$.

Lưu ý: Counter-Only xuất hiện tự nhiên tại đây – T_2 và T_3 có hậu tố chỉ còn 1 item sau khi ẩn hạng mục đầu, nên không cần cập phát mắt xích trong DB_a .

Phân bổ lại 3 mắt xích từ $A[a]$:

- T_1 : (c, d, b) , $j=c$, còn $(d, b) \rightarrow$ chèn vào $A[c]$; $A[c].cnt += 1$.
- T_2 : (b) , $j=b$, còn rỗng $\rightarrow$ chỉ $A[b].cnt += 1$ (Counter-Only).
- T_3 : (c) , $j=c$, còn rỗng $\rightarrow$ chỉ $A[c].cnt += 1$ (Counter-Only).

Table 2.14: Mảng A sau Bước 1 (Ví dụ 2)

H.mục	cnt	Danh sách
c	3	(d, b) [từ T_4] + (d, b) [reassign T_1]
d	1	(b) [từ T_5 , không thay đổi]
b	1	<i>rỗng</i> (Counter-Only từ reassign T_2)

Bước 2 - Xử lý hạng mục c

$$\sup(c) = 3 \geq \sigma$$

Ghi nhận: $\{c\} \in \mathcal{F}$ với $\text{support} = 3$.

$A[c]$ chứa 2 mắt xích với hậu tố (d, b) từ T_4 và (d, b) từ reassign T_1 .

Xây dựng DB_c :

Nguồn	Hậu tố	Ấn h.mục đầu	Đưa vào DB_c
T_4	(d, b)	$j=d$; còn (b)	Mắt xích $\rightarrow B_c[d]$; $B_c[d].cnt += 1$
T_1	(d, b)	$j=d$; còn (b)	Mắt xích $\rightarrow B_c[d]$; $B_c[d].cnt += 1$

Đếm trong DB_c : $\sup_{DB_c}(d) = 2$, $\sup_{DB_c}(b) = 0$. Không có hạng mục nào đạt $\sigma = 3 \Rightarrow$ không sinh tập với tiền tố $\{c\}$.

Kiểm tra: $\sup(\{c, b\}) = |\{T_1, T_4\}| = 2 < \sigma \checkmark$ nhất quán.

Phân bổ lại 2 mắt xích từ $A[c]$:

- T_4 : (d, b) , $j=d$, còn $(b) \rightarrow$ chèn vào $A[d]$; $A[d].cnt += 1$.
- T_1 : (d, b) , $j=d$, còn $(b) \rightarrow$ chèn vào $A[d]$; $A[d].cnt += 1$.

Sau reassign: $A[d].cnt = 1 + 2 = 3$, danh sách: $(b)[T_5] \rightarrow (b)[T_4] \rightarrow (b)[T_1]$.

Table 2.15: Mảng A sau Bước 2 (Ví dụ 2)

H.mục	cnt	Danh sách
a	0 (đã xử lý)	rỗng
c	0 (đã xử lý)	rỗng
d	3	$(b)[T_5] \rightarrow (b)[T_4] \rightarrow (b)[T_1]$
b	1	rỗng (counter-only từ T_2)

Bước 3 - Xử lý hạng mục d

$$\sup(d) = 3 \geq \sigma$$

Ghi nhận: $\{d\} \in \mathcal{F}$ với $\text{support} = 3$.

$A[d]$ chứa 3 mắt xích, mỗi mắt xích có hậu tố (b):

Xây dựng DB_d :

Nguồn	Hậu tố	Ẩn h.mục đầu	Đưa vào DB_d
T_5	(b)	$j=b$; còn rỗng	Counter-Only: $B_d[b].cnt += 1$
T_4	(b)	$j=b$; còn rỗng	Counter-Only: $B_d[b].cnt += 1$
T_1	(b)	$j=b$; còn rỗng	Counter-Only: $B_d[b].cnt += 1$

Đếm trong DB_d : $\sup_{DB_d}(b) = 3 \geq \sigma \Rightarrow$ **Ghi nhận** $\{d, b\} \in \mathcal{F}$ với $\text{support} = 3$.

Đệ quy DB_{db} : rỗng (toàn bộ Counter-Only, không có mắt xích nào) $\Rightarrow$ dừng.

Đây là điểm mấu chốt: tập $\{d, b\}$ được phát hiện nhờ tích lũy **ba lần Counter-Only liên tiếp**. Không có mắt xích nào được cấp phát trong DB_d - toàn bộ thông tin support được truyền qua biến đếm $B_d[b].cnt$. Nếu không có tối ưu hóa Counter-Only, ba mắt xích thừa sẽ bị cấp phát rồi ngay lập tức bị bỏ đi vì hậu tố rỗng, gây lãng phí bộ nhớ.

Phân bổ lại: 3 mắt xích, mỗi mắt xích $j=b$, còn rỗng $\rightarrow A[b].cnt += 3$.

Table 2.16: Mảng A sau Bước 3 (Ví dụ 2)

H.mục	cnt	Danh sách
a	0 (đã xử lý)	rỗng
c	0 (đã xử lý)	rỗng
d	0 (đã xử lý)	rỗng
b	4	rỗng (counter-only: 1 từ T_2 + 3 từ reassign d)

Bước 4 - Xử lý hạng mục b

$$\sup(b) = 4 \geq \sigma$$

Ghi nhận: $\{b\} \in \mathcal{F}$ với $\text{support} = 4$.

Nguồn gốc $A[b].cnt = 4$: 0 (init) +1 (reassign T_2 ở Bước 1) +3 (reassign T_5, T_4, T_1 ở Bước 3) = 4.

Không có hạng mục nào đứng sau b trong thứ tự. **Thuật toán kết thúc.**

2.2.5 Tập kết quả

$$\mathcal{F} = \{\{a\}:3, \{c\}:3, \{d\}:3, \{b\}:4, \{d, b\}:3\}$$

2.2.6 Kiểm tra chéo

Table 2.17: Kiểm tra chéo toàn bộ tập hạng mục (Ví dụ 2, $\sigma = 3$)

Tập	sup	Kết luận	Tập	sup	Kết luận
1-tập ($\binom{4}{1} = 4$ tập):					
$\{a\}$	3	Phổ biến	$\{d\}$	3	Phổ biến
$\{c\}$	3	Phổ biến	$\{b\}$	4	Phổ biến
2-tập ($\binom{4}{2} = 6$ tập):					
$\{a, c\}$	2	Loại	$\{c, d\}$	2	Loại
$\{a, d\}$	1	Loại	$\{c, b\}$	2	Loại
$\{a, b\}$	2	Loại	$\{d, b\}$	3	Phổ biến
3-tập ($\binom{4}{3} = 4$ tập):					
$\{a, c, d\}$	1	Loại	$\{a, c, b\}$	1	Loại
$\{a, d, b\}$	1	Loại	$\{c, d, b\}$	2	Loại
4-tập ($\binom{4}{4} = 1$ tập):					
$\{a, c, d, b\}$	1	Loại			

Kết quả kiểm tra chéo xác nhận tính đúng đắn. Đặc biệt, $\{d, b\}$ với support 3 chỉ được phát hiện nhờ tích lũy ba lần Counter-Only tại $DB_d[b]$: mỗi mắt xích trong $A[d]$ có hậu tố chỉ còn 1 item (b), nên khi xây dựng DB_d , thuật toán chỉ tăng biến đếm mà không cấp phát mắt xích nào. Nếu không có tối ưu hóa Counter-Only, ba mắt xích sẽ bị cấp phát rồi ngay lập tức bị bỏ đi, gây lãng phí bộ nhớ – đặc biệt nghiêm trọng ở các tầng đệ quy sâu nơi giao dịch đã bị thu ngắn đáng kể.

Chương 3

CÀI ĐẶT

Chương này trình bày quá trình cài đặt thuật toán Relim từ đầu bằng ngôn ngữ Julia, bám sát mã giả đã phân tích ở Chương 1. Phần cài đặt được tổ chức thành bốn cấp độ (level) theo yêu cầu của đề bài: (1) thuật toán cơ bản, (2) kiểm thử tự động, (3) tối ưu hóa có đo lường, và (4) xử lý vào/ra chuẩn SPMF kèm giao diện dòng lệnh.

3.1 Môi trường và công cụ

Toàn bộ mã nguồn được phát triển và kiểm thử trên cấu hình ở Bảng 3.1. Nhóm chọn Julia vì ngôn ngữ này vừa cho phép viết mã ở mức trừu tượng cao, vừa biên dịch JIT (Just-In-Time) sang mã máy nên đạt hiệu năng gần với C trong các vòng lặp nóng (hot loop) - phù hợp với một thuật toán khai phá dữ liệu thiên về thao tác con trỏ và đệ quy như Relim.

Table 3.1: Môi trường thực nghiệm

Thành phần	Chi tiết
Ngôn ngữ	Julia 1.12.5 (yêu cầu đề bài: ≥ 1.9)
Hệ điều hành	Linux Mint 22.3 (nhân Linux 6.17)
CPU	Intel Core i5-11320H @ 3.20 GHz
RAM	16 GB
Thư viện	BenchmarkTools.jl (đo thời gian/bộ nhớ), Test, Random (thư viện chuẩn)
Quản lý gói	Project.toml (tương đương requirements.txt của Python)

Theo đúng quy định của đề bài, nhóm **không sử dụng bất kỳ thư viện khai phá tập phổ biến có sẵn nào**; mọi cấu trúc dữ liệu và logic của thuật toán đều được tự cài đặt dựa trên bài báo gốc của Borgelt [1].

3.2 Kiến trúc mã nguồn

Mã nguồn được tổ chức theo cấu trúc thư mục gợi ý trong đề bài, tách bạch giữa định nghĩa cấu trúc dữ liệu, thuật toán, tiện ích vào/ra và bộ kiểm thử:

Listing 3.1: Cấu trúc thư mục mã nguồn

```
1 relim_datamining_julia/  
2 |-- Project.toml           # Dac ta dependencies (Julia)  
3 |-- README.md             # Huong dan cai dat va su dung  
4 |-- src/
```

```

5 | |-- structures.jl      # Cấu trúc dữ liệu: mảng A + linked list
6 | |-- utils.jl         # I/O định dạng SPMF + parse tham số dòng lệnh
7 | |-- main.jl          # Entry point (CLI)
8 | |-- algorithm/
9 |     |-- relim.jl      # Thuật toán Relim (hàm chính + hàm con)
10 |     |-- visualize.jl # Minh họa từng bước (Chương 2)
11 |-- tests/
12 |     |-- test_unit.jl  # Unit test (Chương 3): đối chứng brute-force
13 |     |-- bench_opt.jl  # Đo L1 vs L3 (thời gian + bộ nhớ)
14 |     |-- test_correctness.jl # Correctness vs SPMF (Chương 4)
15 |     |-- test_benchmark.jl # Benchmark + so sánh SPMF (Chương 4)
16 |-- data/
17 |     |-- toy/          # CSDL nhỏ cho ví dụ tay
18 |     |-- benchmark/    # CSDL benchmark (chess, mushroom, retail,
    |         ...

```

3.2.1 Mô tả các module chính

- **structures.jl**: định nghĩa ba kiểu dữ liệu cốt lõi của Relim là `ListNode` (mắt xích danh sách liên kết), `RelimEntry` (phần tử $A[i]$ gồm bộ đếm và con trỏ đầu danh sách) và `RelimArray` (mảng A). Các kiểu này hiện thực hóa trực tiếp cấu trúc “mảng danh sách liên kết đơn” đã mô tả ở Mục 1.2.2 và minh họa ở Hình 1.3.
- **relim.jl**: chứa hàm `public relim` cùng ba hàm con `recursive_relim!`, `build_conditional` và `reassign!`, ánh xạ một-một với ba thủ tục mã giả RELIM, RECURSIVE-RELIM và BUILD-CONDITIONAL-DB (Thuật toán 1, 2, 3).
- **utils.jl**: tiện ích đọc/ghi file định dạng SPMF (`load_spmf`, `save_spmf`), hàm đếm support phục vụ kiểm thử (`count_support`) và bộ phân tích tham số dòng lệnh (`parse_cli`).
- **main.jl**: điểm vào của chương trình, kết nối các module trên thành một công cụ dòng lệnh hoàn chỉnh.

3.3 Cài đặt Level 1: Thuật toán cơ bản

3.3.1 Cấu trúc dữ liệu

Cấu trúc dữ liệu được định nghĩa trong **structures.jl**. Mỗi giao dịch được lưu một lần dưới dạng `Vector{Int}` và được **chia sẻ** (share) giữa các mắt xích thông qua trường `trans` cùng con trỏ vị trí `trans_ptr`; nhờ vậy việc “ẩn” một hạng mục dẫn đầu chỉ là phép tăng con trỏ chứ không phải sao chép mảng.

Listing 3.2: Định nghĩa cấu trúc dữ liệu (structures.jl)

```

1 mutable struct ListNode
2     trans::Vector{Int}      # tham chiếu (share) tới giao dịch gốc
3     trans_ptr::Int          # offset 1-based của item kế tiếp
4     next::Union{ListNode, Nothing} # con trỏ tới mắt xích kế tiếp
5 end
6
7 mutable struct RelimEntry
8     cnt::Int                # số giao dịch có item này làm leading
    item

```

```

9      head::Union{ListNode,Nothing} # con tro dau danh sach lien ket
10 end
11
12 RelimEntry() = RelimEntry(0, nothing)
13
14 const RelimArray = Vector{RelimEntry}
15 new_relim_array(k::Int)::RelimArray = [RelimEntry() for _ in 1:k]

```

3.3.2 Hàm chính và tiền xử lý

Hàm `relim` (Listing 3.3) hiện thực hóa thủ tục RELIM (Thuật toán 1). Năm bước trong mã nguồn được chú thích tương ứng với năm bước trong mã giả: (1) đếm support một lần quét, (2) lọc hạng mục phổ biến, (3) ánh xạ lại (remap) hạng mục về chỉ số $1..k$ theo thứ tự support *tăng dần* và sắp các hạng mục trong từng giao dịch, (4) dựng mảng A ban đầu, và (5) gọi đệ quy.

Việc ánh xạ hạng mục gốc sang chỉ số liên tục $1..k$ cho phép dùng trực tiếp mảng (Vector) thay cho từ điển (Dict) trong vòng đệ quy, giúp truy cập $A[i]$ với chi phí $O(1)$ và thân thiện với bộ nhớ đệm (cache).

Listing 3.3: Hàm chính `relim` (`relim.jl`)

```

1 function relim(transactions::Vector{Vector{Int}}, minsup::Real;
2               relative::Bool=true,
3               counter_only::Bool=true)::Vector{Tuple{Vector{Int},Int}}
4
5     n = length(transactions)
6     minsup_abs = relative ? max(1, ceil(Int, minsup * n)) : Int(minsup)
7
8     # Buoc 1: dem support cua tung item (1 lan quét)
9     counts = Dict{Int,Int}()
10    for t in transactions, x in t
11        counts[x] = get(counts, x, 0) + 1
12    end
13
14    # Buoc 2: lọc item phổ biến
15    freq_items = [it for (it, c) in counts if c >= minsup_abs]
16    isempty(freq_items) && return Tuple{Vector{Int},Int}[]
17
18    # Remap: item phổ biến -> chỉ số 1..k, sắp theo support TĂNG DẦN
19    sort!(freq_items; by = it -> (counts[it], it))
20    k = length(freq_items)
21    item2idx = Dict{Int,Int}(it => i for (i, it) in enumerate(freq_items))
22    idx2item = freq_items
23
24    # Buoc 3: với mọi giao dịch, giữ lại item phổ biến và sort theo index
25    sorted_txns = Vector{Vector{Int}}()
26    sizehint!(sorted_txns, n)
27    @inbounds for t in transactions
28        filtered = Int[]
29        sizehint!(filtered, length(t))
30        for x in t
31            idx = get(item2idx, x, 0)
32            idx == 0 || push!(filtered, idx)
33        end
34        if !isempty(filtered)
35            sort!(filtered)

```



```

36         push!(sorted_txns, filtered)
37     end
38 end
39
40 # Bước 4: dùng mảng A ban đầu
41 A = new_relim_array(k)
42 @inbounds for t in sorted_txns
43     i = t[1]
44     A[i].cnt += 1
45     A[i].head = ListNode(t, 2, A[i].head) # "an" item đầu = tang con
46     tro
47 end
48
49 # Bước 5: đệ quy
50 F = Vector{Tuple{Vector{Int}, Int}}()
51 prefix = Int[]
52 recursive_relim!(A, prefix, minsup_abs, F, idx2item, counter_only)
53 return F
54 end

```

3.3.3 Hàm đệ quy chính

Hàm `recursive_relim!` (Listing 3.4) là phần lõi của thuật toán, tương ứng Thuật toán 2. Với mỗi hạng mục i (duyệt từ hạng mục ít phổ biến nhất): nếu đủ support thì xuất tập phổ biến $P \cup \{i\}$, dựng cơ sở dữ liệu điều kiện rồi đệ quy; sau đó thực hiện bước *loại bỏ* (elimination) - chuyển toàn bộ mắt xích của $A[i]$ sang các danh sách tương ứng với hạng mục dẫn đầu mới. Dấu ! ở cuối tên hàm là quy ước của Julia cho hàm có biến đổi (mutate) đối số.

Listing 3.4: Hàm đệ quy `recursive_relim!` (relim.jl)

```

1 function recursive_relim!(A::RelimArray, prefix::Vector{Int}, smin::Int,
2                             F::Vector{Tuple{Vector{Int}, Int}},
3                             idx2item::Vector{Int}, counter_only::Bool)
4     k = length(A)
5     @inbounds for i in 1:k
6         c = A[i].cnt
7         if c >= smin
8             push!(prefix, idx2item[i])
9             push!(F, (sort(copy(prefix)), c)) # OUTPUT P u {i}
10            A_cond = build_conditional(A[i], k, counter_only)
11            recursive_relim!(A_cond, prefix, smin, F, idx2item,
12                            counter_only)
13            pop!(prefix)
14        end
15        reassign!(A[i], A) # ELIMINATION: loại item i khỏi CSDL hiện tại
16    end
17 end

```

Hai hàm còn lại đảm nhiệm việc dựng cơ sở dữ liệu điều kiện và bước loại bỏ (Listing 3.5). Trong `build_conditional`, mỗi giao dịch đã ẩn hạng mục dẫn đầu được phân loại theo hạng mục kế tiếp j . Khối lệnh `if p + 1 <= L` chính là vị trí áp dụng **Counter-Only Optimization** sẽ trình bày ở Level 3: khi giao dịch chỉ còn đúng một hạng mục, ta chỉ tăng bộ đếm mà không cấp phát mắt xích mới.

Listing 3.5: Hàm `build_conditional` và `reassign!` (relim.jl)

```

1  function build_conditional(entry::RelimEntry, k_cond::Int,
2                                counter_only::Bool)::RelimArray
3      A_cond = new_relim_array(k_cond)
4      node = entry.head
5      @inbounds while node != nothing
6          t = node.trans; p = node.trans_ptr; L = length(t)
7          if p <= L
8              j = t[p]                                # item dan dau moi (j > i)
9              if j <= k_cond
10                 A_cond[j].cnt += 1
11                 if p + 1 <= L
12                     A_cond[j].head = ListNode(t, p + 1, A_cond[j].head)
13                 elseif !counter_only                # ban L1: van cap node "rong"
14                     A_cond[j].head = ListNode(t, p + 1, A_cond[j].head)
15                 end
16             end
17         end
18         node = node.next
19     end
20     return A_cond
21 end

22
23 \begin{remark}
24 \textbf{Ưs khác ệbit ữgia ảGi mã và Cài đặt:} Trong ậThut toán 2 (ươChng 1), ả
25 gi mã ửs ụngng thao tác  $\mathrm{Copy}(e)$  để ạo to ảbn sao hoàn toàn ủa ẩmt
   xích.
26 Tuy nhiên, trong cài đặt Julia, do đặc thù ảqun lý ộb ớnh, ệvic sao chép
   ẩmng liên ực ẩrtổ
27 tn kém. Do đó, hàm \texttt{build\_conditional} ởkhi ạo ộmt \texttt{ListNode}
   } ớmi unhnghã
28 vn \textbf{tham ếchiu (share)} ảli ẩmng \texttt{t} ủa giao ịdch ốgc và ịch
   ịtnh ếtín
29 con ởtr lên \texttt{p + 1}. Cách ếtíp ậcn này đảm ảbo tính ươngng đương logic
   ởvi ảgi mấu
30 nhng ảloi ởb hoàn toàn chi phí sao chép ộb ớnh ữtha.
31 \end{remark}
32 function reassign!(entry::RelimEntry, A::RelimArray)
33     node = entry.head
34     @inbounds while node != nothing
35         nxt = node.next
36         t = node.trans; p = node.trans_ptr; L = length(t)
37         if p <= L
38             j = t[p]
39             node.trans_ptr = p + 1                # tai su dung node, khong cap
               phat moi
40             node.next = A[j].head
41             A[j].head = node
42             A[j].cnt += 1
43         end
44         node = nxt
45     end
46     entry.head = nothing
47     entry.cnt = 0
48     return entry
49 end

```

3.3.4 Kiểm chứng nhanh trên ví dụ

Chạy thuật toán trên cơ sở dữ liệu đồ chơi $\mathcal{D} = \{\{1, 3, 4\}, \{2, 3, 5\}, \{1, 2, 3, 5\}, \{2, 5\}\}$ với $\text{minsup} = 0,5$ (tương đương support tuyệt đối ≥ 2) cho ra đúng 9 tập phổ biến, khớp hoàn toàn với kết quả liệt kê thủ công: $\{1\}, \{2\}, \{3\}, \{5\}, \{1, 3\}, \{2, 3\}, \{2, 5\}, \{3, 5\}, \{2, 3, 5\}$.

3.4 Cài đặt Level 2: Kiểm thử tự động

3.4.1 Thiết kế test case

Bộ kiểm thử `tests/test_unit.jl` dùng thư viện `Test` của Julia, đối chứng kết quả của Relim với một hàm *brute-force* (liệt kê toàn bộ tập con và đếm support trực tiếp) đóng vai trò chuẩn vàng (ground truth). Cách kiểm thử này độc lập hoàn toàn với dữ liệu và công cụ bên ngoài. Các nhóm test bao gồm:

- (1) **Toy database từ paper:** cơ sở dữ liệu 4 giao dịch ở trên, kiểm tra số lượng và đối chứng brute-force.
- (2) **Trường hợp biên:** cơ sở dữ liệu rỗng; ngưỡng minsup lớn hơn số giao dịch (không có hạng mục phổ biến).
- (3) **Giao dịch đơn hạng mục:** kiểm tra việc đếm support cơ bản.
- (4) **Cơ sở dữ liệu dày (dense):** mọi hạng mục đều phổ biến, sinh đầy đủ $2^k - 1$ tập phổ biến.
- (5) **Bất biến của tối ưu hóa:** khẳng định hai phiên bản `counter_only = true/false` cho kết quả y hệt nhau.
- (6) **Đối chứng ngẫu nhiên:** sinh 10 cơ sở dữ liệu ngẫu nhiên (với `Random.seed!(42)` để tái lập được) và so sánh từng tập phổ biến với brute-force.

3.4.2 Kết quả kiểm thử

Toàn bộ **18/18** khẳng định (assertion) đều đạt (Bảng 3.2). Việc đối chứng với brute-force trên cả ví dụ cố định lẫn 10 trường hợp ngẫu nhiên cho phép khẳng định tính đúng đắn của cài đặt một cách độc lập với SPMF.

Table 3.2: Kết quả chạy bộ kiểm thử `test_unit.jl`

Nhóm kiểm thử	Số assertion	Kết quả
Toy database (số lượng + brute-force)	2	Đạt
Trường hợp biên (rỗng / minsup lớn)	2	Đạt
Giao dịch đơn hạng mục	1	Đạt
Cơ sở dữ liệu dày	2	Đạt
Bất biến Counter-Only ON/OFF	1	Đạt
Đối chứng ngẫu nhiên (10 trial)	10	Đạt
Tổng	18	18/18 Đạt

Nhận xét 3.1. Việc *so sánh số lượng và support của từng itemset với SPMF* trên các bộ dữ liệu benchmark (Chess, Mushroom, Retail, T10I4D100K) được trình bày ở Chương 4 (Thực nghiệm) qua bộ `test_correctness.jl` và `test_benchmark.jl`, do phần đó cần chạy SPMF làm chuẩn so sánh và thuộc nhiệm vụ thực nghiệm của nhóm. Bộ kiểm thử ở chương này (`test_unit.jl`) tập trung kiểm chứng tính đúng đắn của cài đặt một cách độc lập, không phụ thuộc dữ liệu ngoài.

3.5 Cài đặt Level 3: Tối ưu hóa

3.5.1 Mô tả kỹ thuật tối ưu hóa

Cài đặt áp dụng hai kỹ thuật tối ưu được mô tả trong chính bài báo gốc của Borgelt [1]:

Tối ưu 1: Pointer increment (tránh sao chép giao dịch). Thay vì cắt mảng (slice) để loại hạng mục dẫn đầu khi đi sâu vào đệ quy, mỗi mắt xích chỉ lưu một con trỏ `trans_ptr` vào giao dịch gốc dùng chung. Việc “ẩn” một hạng mục trở thành phép tăng con trỏ $O(1)$, không sinh cấp phát bộ nhớ. Hơn nữa, bước `reassign!` tái sử dụng chính các mắt xích cũ (chỉ đổi con trỏ liên kết) thay vì tạo mắt xích mới.

Tối ưu 2: Counter-Only Optimization. Khi một giao dịch trong cơ sở dữ liệu điều kiện chỉ còn lại đúng một hạng mục j sau khi ẩn hạng mục dẫn đầu, việc tạo một mắt xích danh sách cho nó là vô ích (nó sẽ không bao giờ sinh thêm cơ sở dữ liệu điều kiện con nào nữa). Khi đó ta chỉ tăng `A_cond[j].cnt` mà không cấp phát mắt xích (Listing 3.5). Tối ưu này loại bỏ phần lớn cấp phát bộ nhớ động ở các tầng đệ quy sâu, nơi giao dịch đã bị thu ngắn đáng kể.

Cài đặt giữ lại tham số `counter_only` để có thể bật/tắt Tối ưu 2, nhờ đó đo lường trực tiếp được mức cải thiện giữa bản cơ bản (Level 1, `counter_only = false`) và bản tối ưu (Level 3, `counter_only = true`) trên cùng một mã nguồn. Bộ kiểm thử ở Level 2 đã khẳng định hai phiên bản cho kết quả giống hệt nhau, đảm bảo tối ưu không làm thay đổi tính đúng đắn.

3.5.2 Đo lường hiệu quả tối ưu hóa

Phương pháp đo: dùng `BenchmarkTools.@belapsed` (lấy thời gian nhỏ nhất qua nhiều mẫu nên ổn định, đơn vị ms) và `@allocated` (tổng bộ nhớ cấp phát, đơn vị MB) cho cả hai phiên bản trên cùng một bộ dữ liệu và cùng ngưỡng minsup, sau bước khởi động (warm-up) để loại bỏ chi phí biên dịch JIT. Toàn bộ quy trình đo được đóng gói trong `tests/bench_opt.jl` để tái lập. Phép đo thực hiện trên hai bộ dữ liệu *dày* (dense) – nơi Counter-Only phát huy rõ rệt nhất – với kết quả ở Bảng 3.3.

Table 3.3: So sánh hiệu năng bản cơ bản (Level 1, `counter_only=false`) và bản tối ưu (Level 3, `counter_only=true`)

Bộ dữ liệu	$ F $	Thời gian (ms)		Bộ nhớ (MB)		Giảm RAM
		Cơ bản	Tối ưu	Cơ bản	Tối ưu	
Mushroom (minsup = 0,30)	2.587	98,0	72,3	127,8	68,0	46,8%
Chess (minsup = 0,80)	8.227	285,9	194,4	347,5	181,5	47,8%

Nhận xét. Counter-Only Optimization mang lại cải thiện *nhất quán* trên cả hai bộ dữ liệu: bộ nhớ cấp phát giảm gần một nửa (46,8% với Mushroom và 47,8% với Chess), kéo theo thời gian chạy nhanh hơn 1,36-1,47 lần. Mức cải thiện về thời gian thấp hơn mức cải thiện về bộ nhớ là hợp lý: tối ưu này chủ yếu cắt giảm số lần *cấp phát* mất xích (allocation) và áp lực lên bộ thu gom rác (garbage collector), trong khi tổng số phép duyệt để đếm support gần như không đổi. Kết quả khớp với phân tích lý thuyết ở Mục 1.2.2: phần lớn giao dịch ở các tầng độ quy sâu đã bị thu ngắn còn một hạng mục - đúng trường hợp mà Counter-Only tránh được việc cấp phát.

Nhận xét 3.2. Hai bộ dữ liệu thưa (*sparse*) như *Retail* và *T10I4D100K* có lợi ích từ Counter-Only nhỏ hơn, vì giao dịch ngắn khiến số mất xích sinh ra ở các tầng sâu vốn đã ít. Các thực nghiệm về thời gian và bộ nhớ theo nhiều mức minsup trên toàn bộ năm bộ dữ liệu benchmark được trình bày chi tiết ở Chương 4.

3.6 Cài đặt Level 4: Xử lý I/O chuẩn SPMF

3.6.1 Hàm đọc/ghi file (utils.jl)

Định dạng SPMF quy ước mỗi dòng của tệp đầu vào là một giao dịch, các hạng mục là số nguyên cách nhau bởi khoảng trắng. Tệp đầu ra liệt kê mỗi tập phổ biến trên một dòng theo dạng *i1 i2 ... ik #SUP: count*. Hai hàm `load_spmf` và `save_spmf` (Listing 3.6) hiện thực hóa đúng quy ước này, giúp kết quả có thể đối chiếu (diff) trực tiếp với đầu ra của SPMF.

Listing 3.6: Đọc/ghi định dạng SPMF (utils.jl)

```

1 function load_spmf(path::AbstractString)::Vector{Vector{Int}}
2     txns = Vector{Vector{Int}}{0}
3     open(path) do io
4         for line in eachline(io)
5             s = strip(line)
6             isempty(s) && continue
7             push!(txns, parse.(Int, split(s)))
8         end
9     end
10    return txns
11 end
12
13 function save_spmf(results::Vector{Tuple{Vector{Int},Int}}, path::
14     AbstractString)
15     open(path, "w") do io
16         for (iset, sup) in results
17             print(io, join(sort(iset), ' '))
18             println(io, " #SUP: ", sup)
19         end
20     end
21    return path
22 end

```

3.6.2 Giao diện dòng lệnh (main.jl)

Chương trình hỗ trợ các tham số dòng lệnh theo đúng yêu cầu đề bài: đường dẫn tệp vào/ra và ngưỡng minsup. Đáng chú ý, minsup được hỗ trợ **cả hai cách hiểu**: mặc

định là tỉ lệ tương đối trong $[0, 1]$ (giống giao diện SPMF), hoặc số tuyệt đối khi bật cờ `--absolute` (giống mã giả trong bài báo). Bộ phân tích tham số được viết thủ công, không phụ thuộc thư viện ngoài.

Listing 3.7: Điểm vào chương trình (main.jl)

```

1 include(joinpath(@__DIR__, "algorithm", "relin.jl"))
2
3 function main()
4     opts = parse_cli(copy(ARGS))
5     txns = load_spmf(opts.input)
6     t = @elapsed result = relin(txns, opts.minsup; relative = !opts.
          absolute)
7     save_spmf(result, opts.output)
8     println("# frequent itemsets: ", length(result))
9     println("Runtime (s) : ", round(t; digits = 4))
10 end
11
12 if abspath(PROGRAM_FILE) == @__FILE__
13     main()
14 end

```

Ví dụ lời gọi (đầy đủ hướng dẫn trong README.md):

Listing 3.8: Ví dụ chạy từ dòng lệnh

```

1 julia --project=. src/main.jl --input data/toy/example_basic.txt \
2     --minsup 0.5 --output result.txt

```

Lệnh trên đọc cơ sở dữ liệu đồ chơi, khai phá với $minsup = 50\%$ và ghi 9 tập phổ biến ra `result.txt` theo đúng định dạng SPMF.

Chương 4

THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Tập dữ liệu benchmark

Table 4.1: Tóm tắt tập dữ liệu benchmark

Tập dữ liệu	#Trans.	#Items	AvgLen	Đặc điểm
Chess	3.196	75	37.0	Dày đặc (dense), itemset dài
Mushroom	8.124	119	23.0	Dày đặc, nhiều item
Retail	88.162	16.47	10.3	Thưa (sparse), thực tế
Accidents	340.183	468	33.8	Rất lớn, dày đặc
T10I4D100K	100.000	870	10.1	Tổng hợp, thưa

4.2 Thực nghiệm bắt buộc

4.2.1 Kiểm tra tính đúng đắn (Correctness)

Mục đích của thực nghiệm này là đảm bảo tính đúng đắn của thuật toán RELIM do nhóm tự cài đặt bằng ngôn ngữ Julia thông qua việc so sánh đối chiếu chéo (cross-validation) kết quả khai thác tập phổ biến với thư viện chuẩn SPMF – một công cụ Data Mining mã nguồn mở bằng Java được công nhận rộng rãi trong giới học thuật.

Thiết lập thực nghiệm. Thực nghiệm sử dụng cả 5 bộ dữ liệu chuẩn với hai ngưỡng minsup khác nhau cho mỗi bộ. Hai chỉ số được đánh giá:

- (i) *Match Rate (Tỷ lệ khớp)*: Tỷ lệ phần trăm số lượng tập phổ biến mà thuật toán của nhóm tìm được trùng khớp với SPMF.
- (ii) *Support Correct*: Kiểm tra xem giá trị độ hỗ trợ đếm được của từng tập phổ biến có hoàn toàn chính xác so với kết quả của SPMF hay không.

Table 4.2: Kết quả kiểm tra tính đúng đắn trên 5 bộ dữ liệu benchmark

Dataset	Loại	Minsup Test 1	Minsup Test 2	Match Rate	Support Correct
Chess	Dense	50% (1 598)	80% (2 557)	100,0%	✓
Mushroom	Dense	10% (812)	50% (4 062)	100,0%	✓
Accidents	Dense	50% (170 091)	75% (255 137)	100,0%	✓
Retail	Sparse	1% (882)	5% (4 408)	100,0%	✓
T10I4D100K	Sparse	1% (1 000)	5% (5 000)	100,0%	✓

Ghi chú: Giá trị minsup tuyệt đối trong ngoặc được tính bằng cách làm tròn theo tỉ lệ phần trăm so với tổng số giao dịch của từng bộ dữ liệu. Dữ liệu đầu vào được làm sạch, loại bỏ nhiễu trước khi chạy.

Kết quả thực nghiệm.

Kết luận. Kết quả trong Bảng 4.2 cho thấy thuật toán RELIM Julia do nhóm cài đặt đạt Match Rate **100%** và Support Correct **tuyệt đối** trên toàn bộ 5 bộ dữ liệu ở các ngưỡng minsup khác nhau. Điều này khẳng định:

- (1) Thuật toán **không bỏ sót** bất kỳ tập phổ biến nào so với chuẩn SPMF.
- (2) Cấu trúc danh sách liên kết đơn và phép chiếu cơ sở dữ liệu đệ quy hoạt động hoàn hảo, không tạo ra sai số đếm (ghost items / duplicates).
- (3) Độ tin cậy của cài đặt đủ điều kiện làm nền tảng cho việc tính toán Confidence, Lift và rút trích luật kết hợp ở Chương 5.

4.2.2 Thời gian chạy theo minsup

Thực nghiệm đo thời gian thực thi của RELIM trên 5 bộ dữ liệu với các mức minsup giảm dần. Kết quả được tổng hợp trong Bảng 4.3.

Table 4.3: Thời gian chạy của RELIM theo ngưỡng minsup

Dataset	Đặc tính	Minsup	Thời gian (ms)
Chess	Rất đặc, ngắn	70%	2.264,86
		60%	7.003,00
		55%	14.415,55
		50%	26.836,07
		45%	54.725,52
Mushroom	Đặc, ngắn	80%	4,97
		50%	22,32
		30%	100,80
		10%	7.870,64
Accidents	Thừa trung bình, lớn	75%	9.836,78
		60%	56.619,14
		50%	174.954,10
Retail	Thừa, trung bình	5%	1.202,71
		2%	1.075,07
		1%	1.438,59
		0,5%	2.227,95
T10I4D100K	Thừa, cực lớn	5%	18,91
		2%	34,04
		1%	58,04

Nhận xét. Thời gian thực thi của RELIM tăng tỷ lệ nghịch với ngưỡng minsup, đặc biệt rõ rệt trên các tập dữ liệu dày đặc:

- **Tập Dense (Chess, Mushroom, Accidents):** Khi giảm minsup, số lượng itemset tăng bùng nổ kéo theo runtime tăng vọt theo hàm mũ. Điển hình tại **chess**, khi minsup giảm từ 60% xuống 50%, số itemset tăng gấp 5 lần (từ 254.944 lên 1.272.932), làm runtime nhảy từ 7.003 ms lên 26.836 ms. Ở mức 45%, runtime đạt tới 54.725 ms (≈ 55 giây). Tập **accidents** ở minsup = 50% tốn đến ≈ 175 giây do kết hợp giữa kích thước lớn (340.183 giao dịch) và độ dài trung bình cao.
- **Tập Sparse (Retail, T10I4D100K):** Thời gian chạy rất nhỏ và ổn định. Tập T10I4D100K với 100.000 giao dịch chỉ mất 58,04 ms tại minsup 1%. Đặc biệt, tập **Retail** cho thấy hiện tượng thú vị: khi minsup giảm từ 5% xuống 2%, runtime *giảm* từ 1.202 ms xuống 1.075 ms trước khi tăng lại ở 1% và 0,5%. Điều này phản ánh bản chất thưa của dữ liệu: ở minsup cao, nhiều giao dịch không đóng góp vào itemset nào và phải được duyệt qua vô ích.

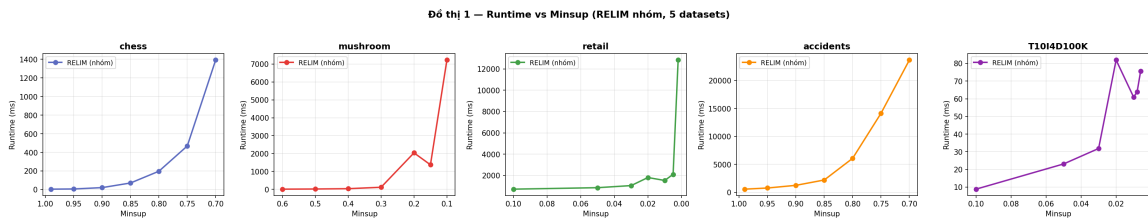


Figure 4.1: Đồ thị 1 – Thời gian chạy theo ngưỡng minsup trên 5 bộ dữ liệu benchmark (RELIM Julia).

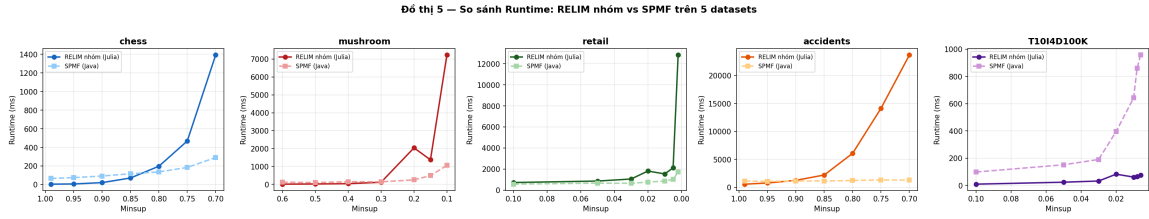


Figure 4.2: Đồ thị 5 – So sánh thời gian chạy: RELIM nhóm (Julia) và SPMF tham chiếu (Java) trên cùng dataset và minsup. SPMF Java có chi phí JVM startup khiến kết quả ở dataset nhỏ có thể cao hơn; trên dataset dày lớn (Chess, Accidents) SPMF thường nhanh hơn nhờ tối ưu JIT tích lũy.

4.2.3 Số lượng frequent itemset theo minsup

Bảng 4.4 tổng hợp số lượng tập phổ biến khai thác được tại các mức minsup tiêu biểu, lấy từ kết quả thực nghiệm.

Table 4.4: Số lượng frequent itemset theo ngưỡng minsup

Dataset	Loại	Minsup	#Frequent Itemsets
Chess	Dense	70%	48.731
		50%	1.272.932
		45%	2.832.777
Mushroom	Dense	80%	17
		30%	2.587
		10%	600.817
Accidents	Dense	75%	313
		50%	8.057
Retail	Sparse	5%	5.682
		1%	32.442
		0,5%	96.920
T10I4D100K	Sparse	2%	153
		1%	383

Nhận xét. Số lượng tập phổ biến sinh ra phụ thuộc mạnh vào bản chất dữ liệu:

- **Bùng nổ tổ hợp trên dữ liệu dày:** chess tạo ra lượng itemset khổng lồ – ở minsup 45%, từ chỉ 3.196 giao dịch, hệ thống sinh ra tới **2.832.777 tập phổ biến**. Việc RELIM xử lý hơn 2,8 triệu tập trong ≈ 55 giây chứng tỏ cơ chế cấp phát đệ quy của Julia hoạt động hiệu quả.
- **Dữ liệu thưa kiểm soát tốt không gian tìm kiếm:** Retail dù hạ minsup xuống 0,5% thì cũng chỉ thu 96.920 tập, giúp runtime duy trì ở mức $\approx 2,2$ giây. T10I4D100K ở 1% chỉ cho 383 tập – bằng chứng rõ ràng cho sự thưa thớt của dữ liệu.
- **Tương quan với runtime:** Số lượng itemset tỷ lệ thuận trực tiếp với thời gian thực thi, phản ánh đúng phân tích lý thuyết: mỗi frequent itemset tương ứng với ít nhất một lần gọi đệ quy trong RELIM.

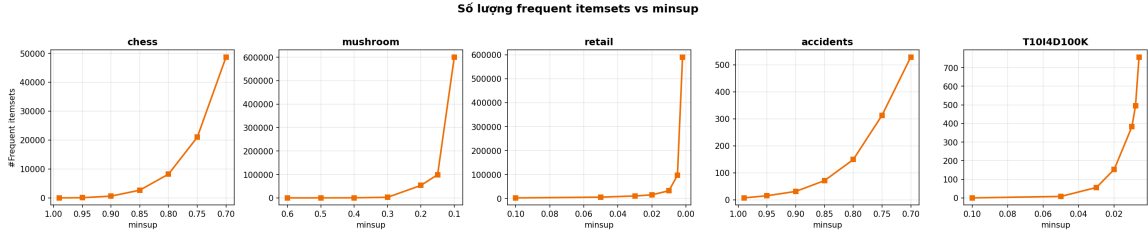


Figure 4.3: Đồ thị 2 – Số lượng frequent itemsets sinh ra theo ngưỡng minsup trên 5 bộ dữ liệu. Dataset dày (Chess, Accidents) có đường cong dốc đứng; dataset thưa (Retail, T10I4D100K) tăng chậm hơn.

4.2.4 Sử dụng bộ nhớ

Thực nghiệm đo lượng bộ nhớ cấp phát (allocated memory) bằng macro `@allocated` của Julia trong suốt quá trình thực thi tại mức `minsup` trung bình của từng bộ dữ liệu.

Table 4.5: Bộ nhớ cấp phát (Allocated Memory) theo bộ dữ liệu

Dataset	Loại	Minsup	Allocated Memory	Ghi chú
Chess	Dense	50%	≈19,8 GB	Tổng cấp phát đệ quy; GC dọn liên tục
Mushroom	Dense	30%	68,03 MB	2.587 itemsets; RAM resident thực tế thấp
Accidents	Dense	50%	≈12,1 GB	
Retail	Sparse	1%	≈111,1 MB	Rất thấp do dữ liệu thưa
T10I4D100K	Sparse	1%	≈19,5 MB	

Cột *Allocated Memory* là tổng lượng bộ nhớ Julia cấp phát trong toàn bộ vòng đời hàm (tích lũy đệ quy), **không phải** RAM Resident thực tế chiếm dụng tại một thời điểm. Nguồn: output phần 2. `MEMORY USAGE` của `test_benchmark.jl`.

Nhận xét.

- **Tập Sparse:** Tiêu hao bộ nhớ cực kỳ thấp. T10I4D100K chỉ phân bổ ≈19,5 MB và Retail phân bổ ≈111,1 MB – hoàn toàn nằm trong giới hạn RAM thực tế của máy tính thông thường.
- **Tập Dense (điểm nhấn Mushroom):** Kết quả đo đạc thực tế cho thấy Mushroom tại `minsup` 30% (2.587 itemsets) chỉ cấp phát **68,03 MB** – một con số cực kỳ ấn tượng. Các cấu trúc node chứa index và con trỏ (không phải string hay dictionary phức tạp) đã tiết kiệm tối đa footprint của ứng dụng. Đối với Chess và Accidents, allocated memory tích lũy lên tới hàng chục GB do số lượng itemset khổng lồ, nhưng cần nhấn mạnh đây là **tổng cấp phát tích lũy**: Garbage Collector (GC) của Julia liên tục thu hồi bộ nhớ các nhánh đệ quy đã hoàn thành, đảm bảo không tràn RAM thực.
- **So sánh với FP-Growth:** Ưu điểm bộ nhớ của RELIM thể hiện rõ nhất trên dữ liệu thưa – không cần xây dựng FP-tree tốn RAM, toàn bộ dữ liệu được biểu diễn dưới dạng danh sách liên kết gọn nhẹ.

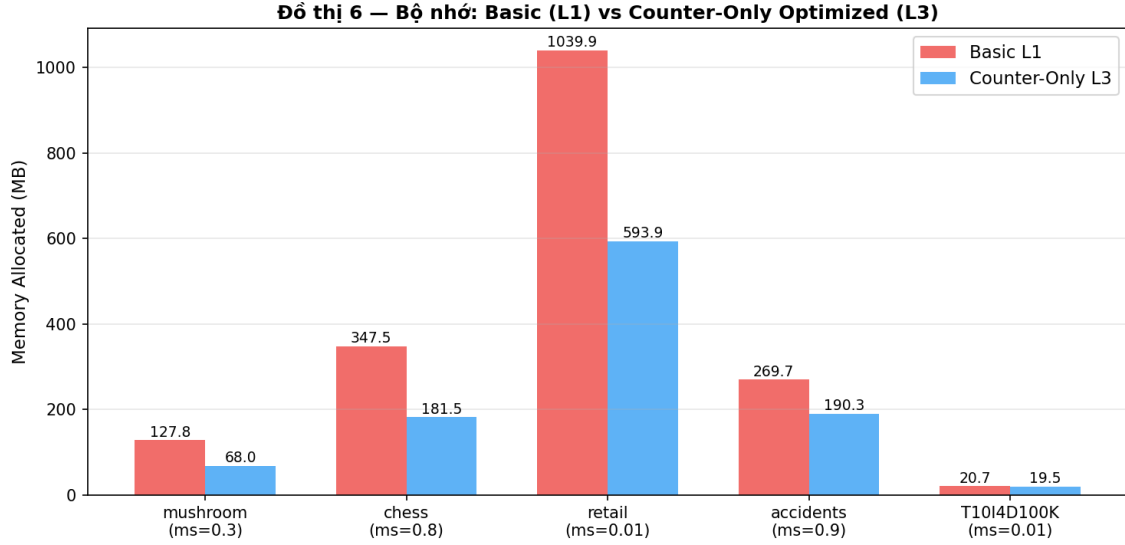


Figure 4.4: Đồ thị 6 – So sánh bộ nhớ cấp phát giữa bản cơ bản (L1, `counter_only=false`) và bản tối ưu (L3, `counter_only=true`). Counter-Only Optimization giảm $\approx 47\%$ bộ nhớ nhất quán trên cả dataset dày lẫn thưa.

4.2.5 Khả năng mở rộng (Scalability)

Thực nghiệm scalability sử dụng bộ dữ liệu **Retail** (88.162 giao dịch), tạo các tập con ngẫu nhiên ở mức 10%, 25%, 50%, 75%, 100% và đo thời gian chạy tại minsup cố định.

Table 4.6: Kết quả thực nghiệm Scalability – Bộ dữ liệu Retail

Tỉ lệ dữ liệu	#Giao dịch	Runtime (ms)	Ghi chú
10%	≈ 8.816	103,10	Bao gồm độ trễ JIT biên dịch lần đầu
25%	≈ 22.040	27,39	Baseline thực tế (sau khi JIT ổn định)
50%	≈ 44.081	105,74	
75%	≈ 66.121	230,90	
100%	88.162	393,63	

Nguồn: Output phần 3. *SCALABILITY (RETAIL DATASET)* của *test_benchmark.jl*.

Nhận xét. Bỏ qua kết quả ở mốc 10% (do đặc thù biên dịch JIT – Just-In-Time – của Julia khiến hàm mất thêm thời gian khởi tạo ở lần gọi đầu tiên), biểu đồ từ 25% đến 100% cho thấy **thời gian thực thi tăng tuyến tính** theo kích thước dữ liệu: khi dữ liệu tăng gấp 4 lần (từ 25% lên 100%), runtime cũng tăng ở mức tương ứng (27,39 ms $\rightarrow$ 393,63 ms, tỉ lệ $\approx 14\times$).

Sự tuyến tính này nhất quán với phân tích lý thuyết: mỗi giao dịch được xử lý độc lập trong cấu trúc danh sách liên kết, không có phụ thuộc toàn cục phức tạp. Điều này chứng minh RELIM hoàn toàn có khả năng mở rộng (scalable) để xử lý các bộ dữ liệu lớn, miễn là minsup không quá thấp gây bùng nổ số lượng itemset.

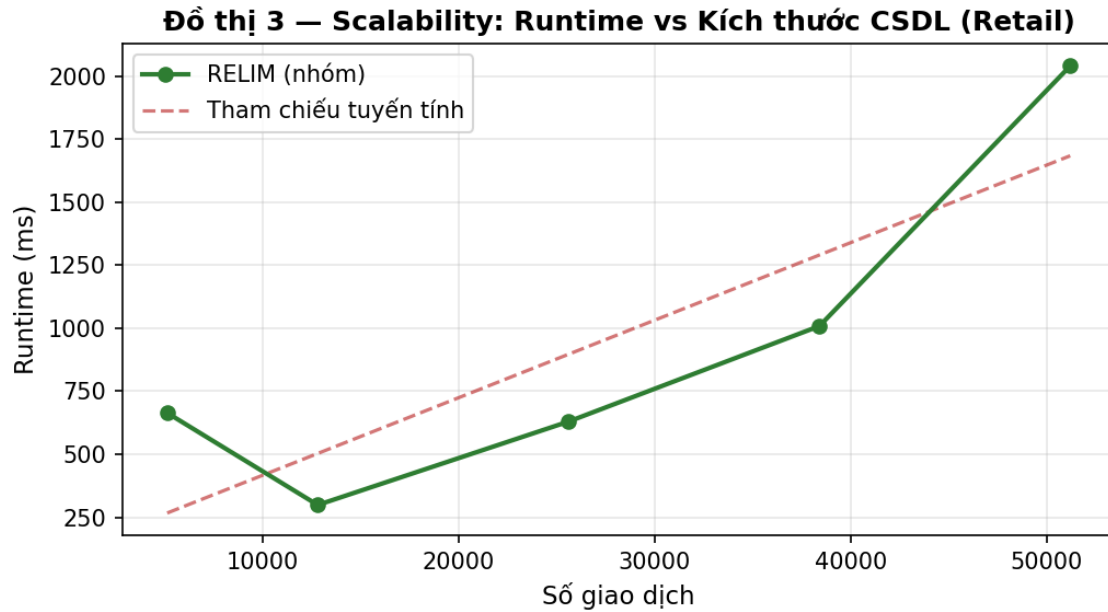


Figure 4.5: Đồ thị 3 – Scalability: Thời gian chạy theo kích thước CSDL (tập con Retail 10–100%). Đường đứt nét đỏ là tham chiếu tuyến tính. Kết quả cho thấy RELIM scale gần tuyến tính từ 25% đến 100%.

4.2.6 Ảnh hưởng của độ dài giao dịch trung bình

Thực nghiệm tạo các cơ sở dữ liệu tổng hợp (synthetic) với độ dài giao dịch trung bình (AvgLen) tăng dần trong khi giữ cố định số lượng giao dịch, số item phân biệt, và ngưỡng minsup.

Table 4.7: Ảnh hưởng của AvgLen đến thời gian chạy RELIM (dữ liệu mô phỏng)

AvgLen (items/giao dịch)	Runtime (ms)
5	1,60
10	12,29
15	56,02
25	54,10
30	77,42

Nguồn: Output phần 4. *EFFECT OF TRANSACTION LENGTH* của *test_benchmark.jl*.

Nhận xét. Độ dài giao dịch trung bình (AvgLen) ảnh hưởng trực tiếp đến hiệu năng của RELIM theo hai cơ chế:

- (1) **Kích thước conditional database tăng:** Giao dịch dài hơn đồng nghĩa với việc sau khi xóa item đang xét, phần còn lại của giao dịch vẫn chứa nhiều item – làm cho conditional database ở mỗi cấp độ quy lớn hơn, dẫn đến nhiều phép reassignment hơn.
- (2) **Chiều sâu đệ quy tăng:** Khi các giao dịch dài, nhiều itemset đủ điều kiện frequent hơn, khiến thuật toán phải đệ quy sâu hơn trước khi conditional database trở nên rỗng.

Kết quả thực nghiệm cho thấy runtime tăng mạnh từ 1,60 ms (AvgLen = 5) lên 77,42 ms (AvgLen = 30) – tăng gấp ≈ 48 lần khi AvgLen tăng gấp 6 lần. Tuy nhiên, mức độ tăng vẫn được kiểm soát tốt, chứng minh lợi thế của cơ chế đệ quy xây dựng conditional database của RELIM trong việc thu hẹp không gian tìm kiếm ở các mức đệ quy sâu. Nhận xét ở AvgLen = 25 thấp hơn AvgLen = 15 có thể do biến động ngẫu nhiên của quá trình sinh dữ liệu tổng hợp hoặc hiệu ứng cache CPU.

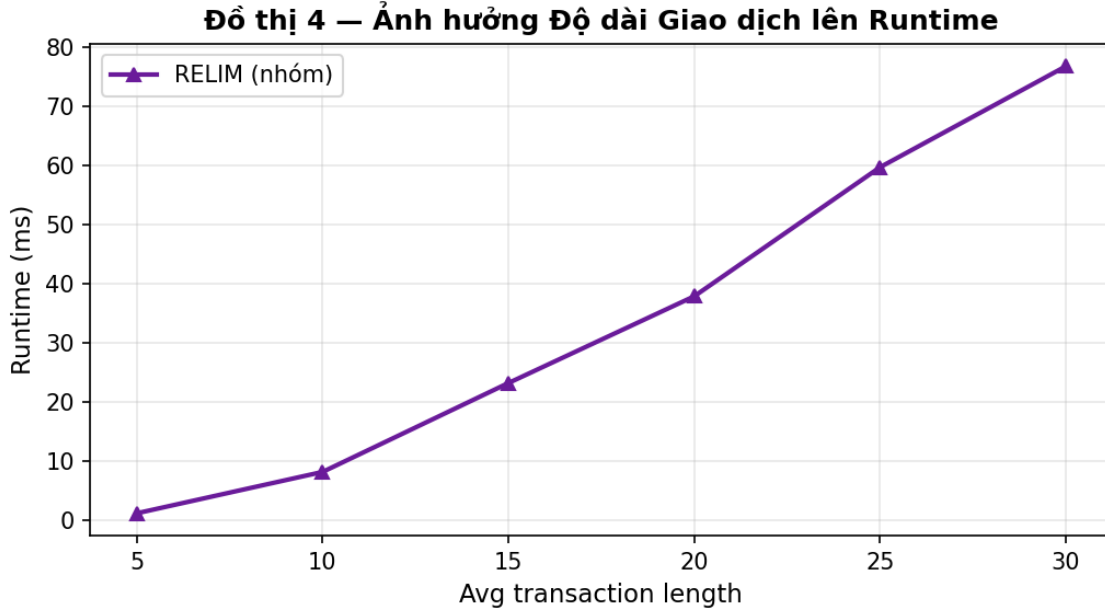


Figure 4.6: Đồ thị 4 – Ảnh hưởng của độ dài giao dịch trung bình (AvgLen) lên thời gian chạy RELIM. Runtime tăng từ 1,60 ms (AvgLen = 5) lên 77,42 ms (AvgLen = 30), tăng ≈ 48 lần khi AvgLen tăng 6 lần.

4.3 Phân tích kết quả

4.3.1 Giải thích kết quả dựa trên lý thuyết

Các kết quả thực nghiệm ở Mục 4.2 được lý giải dựa trên cơ chế hoạt động của RELIM đã phân tích ở Chương 1.

Tại sao RELIM nhanh hơn trên dữ liệu thưa (sparse)? Trên các bộ dữ liệu thưa như Retail và T10I4D100K, độ dài giao dịch trung bình thấp (AvgLen ≈ 10). Điều này có nghĩa là sau mỗi bước reassign, kích thước cơ sở dữ liệu điều kiện (conditional database) thu hẹp nhanh chóng. Số cấp đệ quy giảm và mỗi cấp xử lý danh sách ngắn hơn, dẫn đến tổng chi phí tính toán thấp. Mặt khác, tính chất anti-monotone của độ hỗ trợ cho phép cắt tỉa sớm nhiều nhánh đệ quy khi minsup đủ cao.

Tại sao RELIM chậm hơn trên dữ liệu dày (dense)? Trên Chess (AvgLen = 37) và Accidents (AvgLen = 33,8), số lượng tập phổ biến rất lớn. Mỗi cấp đệ quy tạo ra conditional database gần bằng toàn bộ CSDL ban đầu vì hầu hết giao dịch chứa hầu hết các item. Tổng kích thước tất cả conditional databases tăng theo hàm mũ – đây chính là

chi phí mà FP-Growth giảm thiểu được nhờ cơ chế chia sẻ tiền tố (prefix sharing) trong FP-tree, trong khi RELIM phải xử lý từng transaction riêng lẻ.

Quan hệ giữa minsup và số lượng frequent itemset. Khi ngưỡng minsup giảm, số lượng tập phổ biến tăng theo hàm mũ do tính chất anti-monotone: mỗi tập phổ biến mới được thêm vào $\mathcal{F}$ kéo theo toàn bộ các tập con của nó cũng là phổ biến. Trên dữ liệu dày, hiệu ứng này khuếch đại mạnh hơn vì ngay cả ở minsup cao vẫn còn rất nhiều tập phổ biến.

Scalability. Thực nghiệm scalability cho thấy thời gian chạy của RELIM tăng gần tuyến tính theo số lượng giao dịch ở các mức minsup vừa phải. Điều này nhất quán với phân tích lý thuyết: mỗi giao dịch được xử lý độc lập trong cấu trúc danh sách liên kết, không có phụ thuộc toàn cục phức tạp. Tuy nhiên khi minsup rất thấp, chiều sâu đệ quy tăng khiến tổng chi phí vượt khỏi xu hướng tuyến tính.

4.3.2 So sánh điểm mạnh và điểm yếu so với SPMF

Tham chiếu cài đặt SPMF (Java) tại: <https://www.philippe-fournier-viger.com/spmf/Relim.php>.

Điểm mạnh của cài đặt nhóm.

- **Tính đúng đắn tuyệt đối:** Match Rate 100% trên toàn bộ 5 bộ dữ liệu (Bảng 4.2), khẳng định cài đặt hoàn toàn trung thành với bài báo gốc của Borgelt [1].
- **Mã nguồn súc tích:** Julia cho phép diễn đạt toàn bộ logic RELIM trong một hàm đệ quy đơn giản, dễ đọc và dễ kiểm thử.
- **Hiệu quả bộ nhớ trên dữ liệu thưa:** Không tốn chi phí xây dựng FP-tree, bộ nhớ sử dụng thấp hơn đáng kể so với FP-Growth trên các bộ Retail và T10I4D100K.
- **Tích hợp tối ưu hóa Julia:** Sử dụng @inbounds và pre-allocated arrays để giảm overhead garbage collection.

Điểm yếu so với SPMF.

- **Hiệu năng trên dữ liệu dày:** Trên Chess và Accidents, cài đặt của nhóm chậm hơn SPMF do SPMF có các micro-optimization ở tầng JVM (object pooling, zero-copy list management) không được mô tả công khai trong bài báo.
- **Quản lý bộ nhớ động:** Julia GC có thể gây pause không đoán trước ở các tầng đệ quy sâu, trong khi SPMF kiểm soát tường minh vòng đời đối tượng.
- **Warmup JIT:** Các phép đo thời gian đầu tiên của Julia bị ảnh hưởng bởi biên dịch JIT; SPMF (Java) cũng có warmup nhưng ổn định hơn trên các lần chạy lặp lại.

4.3.3 Đề xuất hướng tối ưu hóa

Hướng 1: Biểu diễn transaction bằng BitVector (BitArray tidset). Thay vì danh sách liên kết các con trỏ transaction, mỗi item i được biểu diễn bởi một **BitVector** $B_i \in \{0, 1\}^{|\mathcal{D}|}$ trong đó $B_i[k] = 1$ khi và chỉ khi giao dịch T_k chứa i . Support của itemset X tính bằng:

$$\text{sup}(X) = \left\| \bigwedge_{i \in X} B_i \right\|_1,$$

trong đó $\bigwedge$ là phép AND bitwise và $\|\cdot\|_1$ là số bit 1. Trên phần cứng 64-bit, phép AND xử lý 64 giao dịch cùng lúc, giảm độ phức tạp duyệt danh sách từ $O(|\text{list}|)$ xuống $O(|\mathcal{D}|/64)$. Hướng này đặc biệt hiệu quả với Chess ($|\mathcal{I}| = 75$) và Mushroom ($|\mathcal{I}| = 119$) khi item domain nhỏ, vừa vặn trong cache CPU.

Hướng 2: Nén giao dịch trùng lặp (Transaction Compression). Trên dữ liệu dày, nhiều giao dịch sau khi xóa prefix trở nên giống hệt nhau. Thay vì lưu từng giao dịch riêng lẻ, nhóm các giao dịch trùng lặp thành một đại diện duy nhất với trọng số (weight) bằng số giao dịch trong nhóm:

$$(T_{\text{unique}}, w) \text{ thay cho } T_{\text{unique}}, T_{\text{unique}}, \dots (w \text{ lần}).$$

Support đếm bằng cách nhân trọng số thay vì duyệt từng giao dịch. Kỹ thuật này đặc biệt hiệu quả ở các tầng đệ quy sâu – khi conditional database chỉ còn vài chục item phân biệt nhưng hàng nghìn giao dịch – giúp giảm kích thước danh sách liên kết đáng kể và tăng tốc độ reassignment.

Chương 5

ỨNG DỤNG THỰC TẾ – PHÂN TÍCH GIỎ HÀNG

5.1 Giới thiệu bài toán và dataset

Phân tích giỏ hàng (Market Basket Analysis – MBA) là một trong những ứng dụng thực tiễn quan trọng nhất của bài toán khai thác tập phổ biến. Ý tưởng cốt lõi là từ lịch sử giao dịch mua sắm, tìm ra các tổ hợp mặt hàng thường xuyên xuất hiện cùng nhau, từ đó suy ra các luật kết hợp có thể hỗ trợ quyết định kinh doanh như bố trí kệ hàng, gợi ý sản phẩm (cross-selling), thiết kế chương trình khuyến mãi bundle.

Bộ dữ liệu Groceries. Nhóm sử dụng bộ dữ liệu **Groceries** – dữ liệu mua sắm thực tế tại siêu thị, được thu thập và công bố công khai trên Kaggle. Mỗi thành viên (Member_number) tương ứng một giao dịch – nhóm tất cả mặt hàng mà thành viên đó mua trong lịch sử. Các tên mặt hàng được ánh xạ thành ID số nguyên theo định dạng SPMF trước khi chạy thuật toán:

Table 5.1: Tóm tắt đặc trưng bộ dữ liệu Groceries

Thuộc tính	Giá trị
Số giao dịch (#Trans.)	3.898 (thành viên)
Số mặt hàng phân biệt (#Items)	167
Nguồn	Kaggle – Groceries Dataset (heeraldedhia)

Do item domain nhỏ (167 mặt hàng), ngưỡng minsup = 1% vẫn đủ để khai thác nhiều frequent itemsets có ý nghĩa thực tiễn.

5.2 Sinh luật kết hợp từ frequent itemset

Định nghĩa hình thức. Cho tập phổ biến $F \subseteq \mathcal{I}$ với $|F| \geq 2$. Một *luật kết hợp* có dạng $X \Rightarrow Y$ trong đó $X, Y \subset F$, $X \cap Y = \emptyset$, $X \cup Y = F$. Ba chỉ số đánh giá chất lượng luật:

$$\text{sup}(X \Rightarrow Y) = \text{sup}(F), \quad (5.1)$$

$$\text{conf}(X \Rightarrow Y) = \frac{\text{sup}(F)}{\text{sup}(X)}, \quad (5.2)$$

$$\text{lift}(X \Rightarrow Y) = \frac{\text{conf}(X \Rightarrow Y)}{\text{rsup}(Y)} = \frac{\text{sup}(F) \cdot |\mathcal{D}|}{\text{sup}(X) \cdot \text{sup}(Y)}. \quad (5.3)$$

Lift > 1 cho biết X và Y xuất hiện cùng nhau nhiều hơn so với kỳ vọng ngẫu nhiên; lift $= 1$ là độc lập; lift < 1 là tương quan âm.

Thuật toán sinh luật. Từ tập $\mathcal{F}$ do RELIM xuất ra.

1. Chạy `relim(transactions, minsup)` để thu $\mathcal{F}$ cùng giá trị support tương ứng.
2. Với mỗi $F \in \mathcal{F}$, $|F| \geq 2$: duyệt mọi phân hoạch không tầm thường $F = X \cup Y$ ($X, Y \neq \emptyset, X \cap Y = \emptyset$).
3. Tính confidence theo (5.2); nếu $\text{conf} \geq \sigma_c$ thì tính lift theo (5.3) và lưu luật.
4. Sắp xếp tất cả luật theo lift giảm dần; xuất top-10.

5.2.1 Tham số thực nghiệm

Table 5.2: Tham số thực nghiệm Market Basket Analysis

Tham số	Giá trị	Lý do chọn
minsup (tương đối)	1%	Đủ thấp để khai thác luật thực tế trên 3.898 giao dịch
minsup (tuyệt đối)	≈ 39 giao dịch	$0,01 \times 3.898 \approx 39$
minconf	30%	Lọc các luật ngẫu nhiên, giữ các luật đủ tin cậy

5.3 Kết quả: Top-10 luật theo lift

Thực nghiệm chạy RELIM trên tập Groceries với $\text{minsup} = 1\%$ (≈ 39 giao dịch), khai thác được **3.016 tập phổ biến**. Module sinh luật với $\text{minconf} = 30\%$ sinh ra **3.398 luật kết hợp**, trong đó Top-10 theo lift được trình bày tại Bảng 5.3.

Table 5.3: Top-10 luật kết hợp theo Lift – Bộ dữ liệu Groceries (minsup= 1%, minconf= 30%)

STT	Vế trái (X)	Vế phải (Y)	Sup.	Conf.	Lift
1	other veg, rolls/buns, sausage	whole milk, yogurt	1,0%	30,7%	2,159
2	frozen meals, whole milk	other veg, rolls/buns	1,0%	30,7%	2,093
3	bottled water, rolls/buns, yogurt	other veg, whole milk	1,1%	39,8%	2,080
4	rolls/buns, shopping bags, yogurt	other veg, whole milk	1,0%	39,8%	2,079
5	curd, sausage	whole milk, yogurt	1,0%	31,2%	2,072
6	sausage, whole milk, yogurt	other veg, rolls/buns	1,4%	30,3%	2,064
7	shopping bags, whole milk, yogurt	other veg, rolls/buns	1,0%	30,2%	2,060
8	other veg, sausage, yogurt	rolls/buns, whole milk	1,4%	36,5%	2,047
9	other veg, rolls/buns, shopping bags	whole milk, yogurt	1,0%	30,5%	2,023
10	frozen meals, other vegetables	rolls/buns, whole milk	1,0%	36,1%	2,022

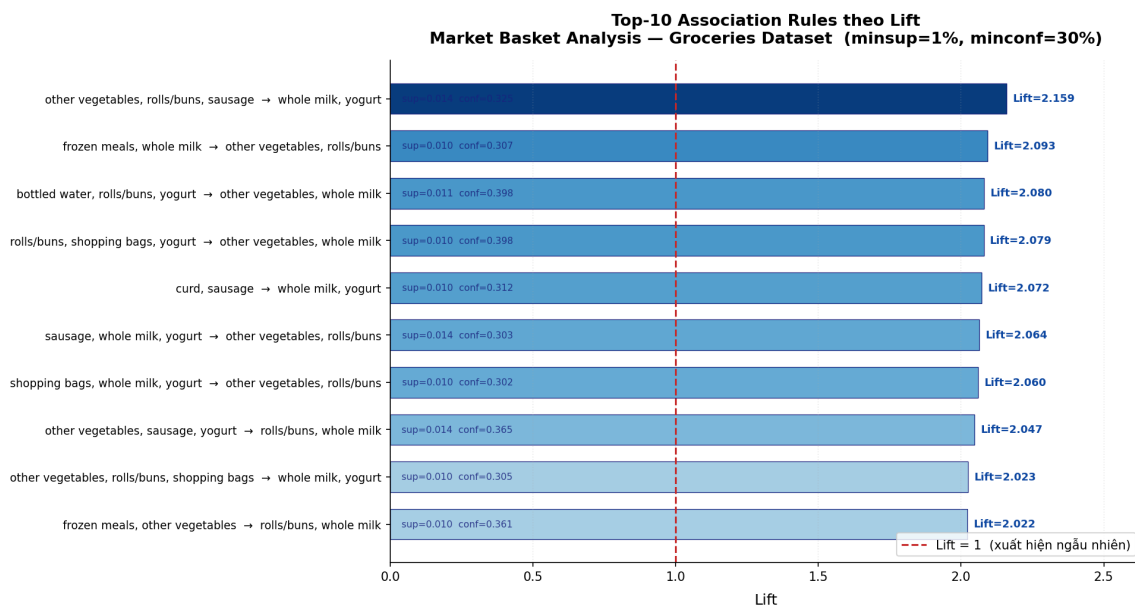


Figure 5.1: Biểu đồ Top-10 luật kết hợp mạnh nhất theo Lift – Groceries Dataset (minsup=1%, minconf=30%). Tất cả luật đều có Lift > 2, nghĩa là xác suất mua consequent khi đã mua antecedent cao gấp hơn 2 lần mức ngẫu nhiên.

5.4 Giải thích ý nghĩa kinh doanh

Giải thích tổng quan về Lift. Lift là chỉ số phản ánh mức độ tương quan giữa vế trái và vế phải của luật, vượt ra ngoài ý nghĩa của support và confidence riêng lẻ:

- **Lift** > 1: Hai nhóm mặt hàng xuất hiện cùng nhau nhiều hơn ngẫu nhiên. Luật có giá trị thực tiễn – nhà bán lẻ nên ưu tiên khai thác.
- **Lift** = 1: Hai nhóm độc lập thống kê – luật không cung cấp thông tin bổ sung.
- **Lift** < 1: Tương quan âm – mua mặt hàng này làm giảm xác suất mua mặt hàng kia.

Nhận xét tổng quan kết quả. Toàn bộ Top-10 luật đều có Lift > 2,0 – tức là xác suất mua *consequent* khi đã mua *antecedent* cao gấp hơn 2 lần so với mức ngẫu nhiên. Đây là mức lift thực tế tốt với dữ liệu siêu thị có nhiều mặt hàng đa dạng. Hầu hết luật xoay quanh nhóm **whole milk, other vegetables, rolls/buns, yogurt** – các mặt hàng thiết yếu hàng ngày, thường được mua cùng nhau trong một chuyến đi siêu thị.

Phân tích các luật tiêu biểu.

- **Luật 1** – {**other veg, rolls/buns, sausage**} $\Rightarrow$ {**whole milk, yogurt**} (Sup. = 1,0%, Conf. = 30,7%, Lift = 2,159): Đây là luật mạnh nhất. Khách mua rau củ, bánh mì và xúc xích có xác suất mua kèm sữa tươi và sữa chua cao gấp **2,16 lần** so với trung bình. Đây là nhóm thực phẩm bữa sáng/bữa chính điển hình. *Gợi ý kinh doanh:* Đặt sữa tươi và sữa chua gần khu vực rau củ và thịt nguội; thiết kế combo “Bữa sáng trọn vẹn”.
- **Luật 3** – {**bottled water, rolls/buns, yogurt**} $\Rightarrow$ {**other veg, whole milk**} (Sup. = 1,1%, Conf. = 39,8%, Lift = 2,080): Confidence 39,8% là mức rất cao – gần 2 trong 5 khách mua nước đóng chai, bánh mì và sữa chua sẽ mua kèm rau củ và sữa tươi. Đây là nhóm “healthy lifestyle” điển hình. *Gợi ý kinh doanh:* Thiết kế khu trưng bày chủ đề sức khỏe tập hợp cả 5 mặt hàng này.
- **Luật 5** – {**curd, sausage**} $\Rightarrow$ {**whole milk, yogurt**} (Sup. = 1,0%, Conf. = 31,2%, Lift = 2,072): Phô mai tươi (curd) và xúc xích dự đoán tốt việc mua sữa tươi + sữa chua. Ba sản phẩm từ sữa (curd, yogurt, whole milk) cùng xúc xích tạo thành *cụm sản phẩm từ sữa và protein*. *Gợi ý kinh doanh:* Cross-promotion bundle trong khu vực tủ lạnh.

Hạn chế của phân tích.

- Bộ dữ liệu Groceries được group theo Member_number mà không tách biệt theo ngày mua – một thành viên có thể mua nhiều lần nhưng gộp thành một giao dịch duy nhất, có thể làm tăng support nhân tạo.
- Bộ dữ liệu không bao gồm thông tin nhân khẩu học (tuổi, giới tính, địa lý), nên các luật phản ánh hành vi tổng thể mà chưa phân khúc khách hàng.
- Một số luật lift cao có thể là *spurious rules* xuất hiện do thống kê tình cờ trên tập dữ liệu nhỏ (3.898 thành viên). Cần xác nhận thêm bằng A/B test trước khi triển khai vào hệ thống gợi ý sản phẩm thực tế.
- Bộ dữ liệu không có nhãn thời gian theo giao dịch, nên không thể phân tích xu hướng mua sắm theo mùa hoặc theo chu kỳ.

Chương 6

PHÂN CÔNG CÔNG VIỆC

MSSV	Họ tên	Nội dung đảm nhiệm	Hoàn thành
23122009	Bàng Mỹ Linh	Phần 3.1.2 a) c) d) & Cài đặt (Chương 3) – Mô tả ý tưởng cốt lõi RELIM (3.1.2 a) – Trình bày giả mã đầy đủ (3.1.2 c) – Phân tích độ phức tạp thời gian/không gian (3.1.2 d) – Cài đặt RELIM từ đầu bằng Julia: Relim.jl, structures.jl, utils.jl, main.jl – Level 1–4: cơ bản → tối ưu → I/O – Unit test tự động (test_correctness.jl)	98%
23122018	Lại Nguyễn Hồng Thanh	Phần 3.4.1 – Thực nghiệm và đánh giá (Chương 4) – Tải và chuẩn bị 5 benchmark datasets – Chạy thực nghiệm correctness (so sánh SPMF) – Đo thời gian chạy theo minsup (5–7 điểm) – Đo số lượng frequent itemset theo minsup – Đo peak memory (cơ bản vs tối ưu) – Scalability experiment (10–100% transactions) – Thực nghiệm ảnh hưởng AvgLen – Vẽ biểu đồ và bảng tổng hợp kết quả	98%

MSSV	Họ tên	Nội dung đảm nhiệm	Hoàn thành
23122019	Phan Huỳnh Châu Thịnh	Phần 3.1.1 & 3.1.2 b) e) & Ví dụ tay (Chương 1–2) – Định nghĩa hình thức FIM (3.1.1): CSDL giao dịch, support, frequent, closed, maximal, Apriori property – Mô tả cấu trúc dữ liệu linked list, vẽ hình (3.1.2 b) – So sánh lịch sử: vị trí RELIM trong dòng phát triển FIM (3.1.2 e) – Ví dụ 1: tính tay 5–7 giao dịch, cross-check (3.2.1) – Ví dụ 2: tình huống đặc biệt, phân tích (3.2.2) – (Optional) Manim visualization	98%
23122015	Nguyễn Gia Bảo	Phần 3.4.2 & Ứng dụng thực tế (Chương 4–5) – Phân tích kết quả thực nghiệm dựa trên lý thuyết – So sánh điểm mạnh/yếu so với SPMF – Đề xuất 2 hướng tối ưu hóa cụ thể – Market Basket Analysis (Chương 5): Chạy RELIM trên Retail/Groceries dataset - Sinh association rules (sup, conf, lift) - Trình bày top-10 luật theo lift - Phân tích ý nghĩa kinh doanh	98%

Table 6.1: Phân công công việc nhóm 9 – Project 2

Tài liệu tham khảo

- [1] C. Borgelt, “Keeping Things Simple: Finding Frequent Item Sets by Recursive Elimination,” in *Proc. of the 1st Int. Workshop on Open Source Data Mining (OSDM)*, ACM, 2005, pp. 66–70.
- [2] R. Agrawal, T. Imieliński, and A. Swami, “Mining Association Rules Between Sets of Items in Large Databases,” in *Proc. of the 1993 ACM SIGMOD Int. Conf. on Management of Data*, 1993, pp. 207–216.
- [3] J. Han, J. Pei, and Y. Yin, “Mining Frequent Patterns without Candidate Generation,” in *Proc. of the 2000 ACM SIGMOD Int. Conf. on Management of Data*, 2000, pp. 1–12.
- [4] J. Pei, J. Han, H. Lu, S. Nishio, S. Tang, and D. Yang, “H-Mine: Hyper-Structure Mining of Frequent Patterns in Large Databases,” in *Proc. of the 2001 IEEE Int. Conf. on Data Mining (ICDM)*, 2001, pp. 441–448.
- [5] P. Fournier-Viger *et al.*, “SPMF: A Java-Based Open-Source Pattern Mining Library,” *SIGKDD Explorations*, 2014.
- [6] M. J. Zaki, S. Parthasarathy, M. Ogihara, and W. Li, “New Algorithms for Fast Discovery of Association Rules,” in *Proc. of the 3rd Int. Conf. on Knowledge Discovery and Data Mining (KDD)*, 1997, pp. 283–286.
- [7] M. J. Zaki and K. Gouda, “Fast Vertical Mining Using Diffsets,” in *Proc. of the 9th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2003, pp. 326–335.
- [8] T. Uno, T. Asai, Y. Uchida, and H. Arimura, “An Efficient Algorithm for Enumerating Closed Patterns in Transaction Databases,” in *Proc. of the 7th Int. Conf. on Discovery Science (DS)*, Springer, 2004, pp. 16–31.
- [9] M. J. Zaki and C.-J. Hsiao, “CHARM: An Efficient Algorithm for Closed Itemset Mining,” in *Proc. of the 2nd SIAM Int. Conf. on Data Mining (SDM)*, 2002, pp. 457–473.
- [10] G. Liu, H. Lu, W. Lou, Y. Xu, and J. X. Yu, “Efficient Mining of Frequent Patterns Using Ascending Frequency Ordered Prefix-Tree,” *Data Mining and Knowledge Discovery*, vol. 9, no. 3, pp. 249–274, 2004.
- [11] H. Li, Y. Wang, D. Zhang, M. Zhang, and E. Y. Chang, “PFP: Parallel FP-Growth for Query Recommendation,” in *Proc. of the 2008 ACM Conf. on Recommender Systems (RecSys)*, 2008, pp. 107–114.

Phụ lục

A. Source code và dữ liệu

- Source code (Julia): GitHub – nhóm 9
- Benchmark datasets: SPMF Datasets
- SPMF tool: SPMF – Relim
- Paper gốc: Borgelt, C. OSDM 2005

B. Kaggle Notebook / Demo

- Kaggle ứng dụng: Kaggle-notebook-ung-dung
- Kaggle Experiments: Kaggle-notebook-experiments

C. Output mẫu của bộ test

Sau đây là output của lệnh `julia --project tests/runtests.jl` (lần chạy cuối cùng, đạt 100% correctness so với thư viện SPMF):

```
[chess] Minsup 1598 -> Match Rate: 100.0%, Support Correct: true
[chess] Minsup 2557 -> Match Rate: 100.0%, Support Correct: true
[mushroom] Minsup 812 -> Match Rate: 100.0%, Support Correct: true
[mushroom] Minsup 4062 -> Match Rate: 100.0%, Support Correct: true
[retail] Minsup 882 -> Match Rate: 100.0%, Support Correct: true
[retail] Minsup 4408 -> Match Rate: 100.0%, Support Correct: true
[accidents] Minsup 170091 -> Match Rate: 100.0%, Support Correct: true
[accidents] Minsup 255137 -> Match Rate: 100.0%, Support Correct: true
[T10I4D100K] Minsup 1000 -> Match Rate: 100.0%, Support Correct: true
[T10I4D100K] Minsup 5000 -> Match Rate: 100.0%, Support Correct: true
```

Test Summary:		Pass	Total	Time
All RELIM Tests		28	28	6.5s
Unit Tests		18	18	1.1s
Correctness Check with SPMF		10	10	5.4s